scientific reports

Check for updates

OPEN

# Spatio-temporal evolutional graph neural network for traffic flow prediction in UAV-based urban traffic monitoring system

Wenming Ma✉, Zihao Chu, Hao Chen & Mingqi Li

**With the advancement of modern UAV technology, UAVs have become integral to creating traffic management monitoring systems. Additionally, UAV-based traffic monitoring systems can predict traffic flow by integrating machine learning methods. Specifically, traffic flow data contains both spatial and temporal information, which can be effectively processed by graph neural networks (GNNs). However, GNNs often face the challenge of oversmoothing, which hinders their ability to capture complex structures in the data. The Spatio-Temporal Graph Ordinary Differential Equations (STGODE) model addresses this issue by introducing Neural Ordinary Differential Equations (NODEs) to construct deeper GNNs. Despite this, STGODE relies on initially predefined semantic neighborhood matrices, which do not adapt well to the dynamic nature of traffic information. To overcome this limitation, we propose an evolutionary graph neural network for traffic prediction, capable of continuously updating the semantic adjacency matrix throughout the training process. This dynamic evolution of the semantic adjacency matrix allows it to adapt to the features and semantic relations of the current data, enhancing its ability to capture the complexity and variability of traffic patterns. We validate our approach through experiments on several real-world datasets, demonstrating that our method outperforms state-of-the-art benchmarks.**

**Keywords** Unmanned aerial vehicle, Traffic monitoring system, Graph neural network, Traffic flow prediction

The rapid pace of urbanization and the continuous increase in automobile ownership have created a significant global challenge: traffic congestion. This pervasive issue not only imposes substantial economic costs and exacerbates environmental degradation but also profoundly affects the well-being of urban populations. As a result, mitigating traffic congestion has become a critical focus of research aimed at addressing these concerns and optimizing transportation systems. Traffic flow prediction is a key area of study that involves forecasting future traffic conditions based on historical traffic data and various relevant factors, such as vehicle speeds, travel times, and congestion levels. Effective management of urban transport networks relies heavily on comprehensive database support, with spatial distribution information of traffic being one of the most crucial components. This information is especially vital in atypical situations, such as road accidents, construction work, vehicle or infrastructure failures, or other incidents that necessitate the redirection of traffic flow.

In such scenarios, traffic data collected by remote sensing equipment over successive time intervals are used to dynamically construct and update the spatial distribution of traffic. This capability is essential for the effective management of urban road networks, enabling the identification of optimal routes in real-time and significantly contributing to the alleviation of traffic congestion. Remote sensing technology has a broad range of applications in traffic management. For instance, sensors and UAVs deployed on roads can monitor vehicle flow, speed, and traffic congestion in real-time, providing critical data support for traffic management and planning. By leveraging these technologies, we can improve the efficiency and effectiveness of urban transportation systems, ultimately enhancing the quality of life in urban areas.

The use of UAVs in traffic forecasting is revolutionizing traditional traffic monitoring and management methods[1]. UAVs can swiftly cover large areas and collect high-resolution traffic data in real-time, including traffic flow, speed, and lane occupancy, providing more accurate and timely inputs for traffic prediction models. Utilizing advanced image processing and machine learning methods, UAVs can automatically detect and classify different vehicle types, identify traffic accidents, and pinpoint congestion areas, enabling traffic

School of Computer and Control Engineering, Yantai University, Yantai 264005, China. ✉email: mwm@ytu.edu.cn

management authorities to take swift actions to optimize traffic flow. Additionally, UAVs can rapidly reach the scene of a traffic accident to provide real-time video and imagery, aiding in quick assessment and decision-making, thereby enhancing accident handling efficiency and reducing secondary incidents and congestion. Data collected by UAVs can be integrated with existing traffic management systems to optimize traffic signal settings, dynamically adjust traffic flow, and predict future traffic patterns. By monitoring the environmental impact of traffic, UAVs support the development of eco-friendly transport policies and reduce traffic-related pollution. Furthermore, UAVs can regularly inspect transport infrastructure, identifying potential safety hazards and improving maintenance efficiency. As a crucial component of Intelligent Transportation Systems (ITS), UAVs achieve comprehensive, multi-level traffic monitoring and management when combined with other sensors and communication technologies. Overall, the application of UAVs[2] in traffic prediction holds significant promise and potential. This approach not only enhances the efficiency and accuracy of data collection but also markedly improves traffic management and accident response capabilities through real-time monitoring and rapid response, providing robust support for the development of intelligent transport systems[3].

The California Department of Transportation utilizes remote sensing technology to collect traffic data over time, resulting in the creation of the Freeway Performance Measurement System (PeMS[4]). This data includes extensive traffic information, such as traffic flow for each road and vehicle speed, making it invaluable for developing traffic flow prediction models. Traffic data is collected over time by sensors installed on the roads, and UAVs gather this information to construct spatiotemporal traffic maps during each data collection. These maps, rich in detailed information, are crucial for operations like traffic prediction. The research presented in this paper is also based on data collected through remote sensing technology.

The importance of accurate traffic flow prediction cannot be overstated, as it forms the foundation of efficient traffic management. By providing transportation authorities with actionable insights, precise traffic flow prediction enables proactive planning, judicious resource allocation, and informed decision-making related to traffic control and system optimization. Essentially, it is crucial for implementing multifaceted strategies aimed at alleviating congestion and promoting a more sustainable urban environment.

In recent years, the rapid development of data science, machine learning, and computational methods has led to significant advancements in traffic flow prediction techniques. Traditionally, traffic prediction relied on time-series analysis models such as Autoregressive Integrated Moving Average (ARIMA[5])and Vector Autoregression (VAR[6]). However, these models often struggle with the complex nonlinear spatiotemporal dynamics of traffic data. To address these limitations, researchers have started using traditional machine learning models like Support Vector Regression (SVR[7]) and k-Nearest Neighbors (KNN[8]), achieving some improvements but still heavily reliant on manually engineered features.

Expanding on the integration of deep learning techniques into traffic prediction methods, recent advancements have witnessed a discernible trend towards the adoption of deep neural networks. These methodologies epitomize a paradigm shift towards a more data-centric approach in dynamic traffic modeling, thereby garnering substantial interest within the research community. Of particular prominence among these approaches is the spatiotemporal graph neural network (STGNN[9]), which has ascended as a frontrunner among the pantheon of frameworks for traffic prediction. Within the STGNN architecture, each node within the graph corresponds to a discrete traffic observation point, while edges delineate the interconnectedness between road traffic nodes, collectively imbuing the model with a network-centric structural framework. Crucially, STGNN harnesses the power of graph convolutional blocks to facilitate spatial processing, synergistically amalgamating this capability with forward computation functions that intricately simulate temporal dynamics. Notably, the efficacy of methods predicated upon STGNN hinges upon the choice of forward computation functions, delineating pathways that diverge into distinct trajectories within the landscape of deep learning. Specifically, these methods bifurcate into two primary categories based on their utilization of recurrent neural networks (RNNs) or convolutional neural networks (CNNs), each conferring unique advantages and exhibiting nuanced performance characteristics.

A salient hallmark of these CNN-based methodologies is their intrinsic capacity to obviate the need for laborious manual feature engineering, circumventing the arduous task of handcrafting features that typifies traditional machine learning paradigms. Instead, these techniques harness the innate power of hierarchical feature learning, thereby endowing them with a heightened aptitude for discerning intricate patterns and latent correlations embedded within the complex fabric of traffic dynamics. Consequently, they engender superior prediction accuracy whilst concurrently mitigating the cognitive overhead associated with feature engineering, thereby constituting a seminal leap forward in the realm of traffic flow forecasting.

However, graph neural network (GNN) models have inherent limitations, particularly when applied to large-scale traffic networks with numerous nodes and edges. Traditional graph convolutional network (GCN) methods encounter computational and storage challenges with such extensive graphs, resulting in relatively shallow networks that struggle to capture long-distance spatial correlations. Neural Ordinary Differential Equations (NODEs[10]) represent a significant advancement in contemporary deep learning methodologies. Unlike conventional neural networks, which have a fixed number of layers and parameters throughout training, NODEs adopt a continuous-time framework, treating data as a dynamic process evolving over time. The key innovation of NODEs is viewing the model's depth as a temporal continuum. Instead of being limited by a rigid architecture with a fixed number of layers, the depth of a NODE model evolves dynamically through the integration of ordinary differential equations (ODEs) over time. This adaptability allows NODEs to flexibly capture the complex temporal dynamics inherent in diverse datasets. By moving beyond the constraints of fixed-depth architectures, NODEs address the issue of oversmoothing commonly found in traditional GNN models. This enables the construction of deeper network architectures and the extraction of richer information embedded within the data.

STGODE[11]was the first to introduce ODE-based methods into spatiotemporal traffic prediction. However, it has limitations in the construction of the semantic adjacency matrix. The method constructs a fixed semantic adjacency matrix before the start of model training, but due to the complexity of traffic flow data, significant dynamic changes occur in both the temporal and spatial dimensions. A fixed semantic adjacency matrix is limited in its ability to capture these changes, potentially leading to suboptimal performance when the model is applied to predict the dynamic variations in traffic flow. This limitation highlights a critical gap in the existing research, underscoring the need for approaches that can dynamically adjust the semantic adjacency matrix to better capture the spatiotemporal variations in traffic flow data and improve prediction accuracy. To address this key issue, we propose an evolutionary graph neural network method, which offers a more effective solution for traffic flow prediction.

STGCN[9] and ASTGCN[12] are commonly used spatiotemporal graph convolutional network models that have achieved significant success in traffic flow prediction. However, both STGCN and ASTGCN rely on fixed semantic adjacency matrices, which present certain limitations in capturing the dynamic spatiotemporal variations in traffic flow data. Specifically, STGCN employs convolution operations to capture neighborhood information in the spatial dimension and uses convolutions to model temporal dynamics. However, its semantic adjacency matrix remains unchanged during training, making it difficult to adapt to the continuously evolving patterns in traffic flow data. ASTGCN attempts to address this issue by introducing spatiotemporal attention mechanisms, yet its semantic adjacency matrix also remains fixed, failing to fully capture the complex spatiotemporal variations inherent in traffic flow data.

To address this issue, we propose an evolutionary graph neural network for traffic prediction that continuously updates the semantic adjacency matrix during the training cycle. This dynamic adjustment process allows the matrix structure to evolve based on the features and semantic relations of the current data. This adaptability enables the model to better handle varying data distributions and feature changes, thereby improving prediction performance and generalization capability. Additionally, the evolving semantic adjacency matrix can promptly capture data changes, allowing the model to adapt more effectively to data distributions across different time points and spatial locations. During the matrix evolution process, we calculate the intersection between the current semantic neighborhood matrix and the initial matrix, ensuring that the evolution retains a degree of historical data information. This helps the model to remember past data while learning from new data, providing a better understanding of the data's evolutionary trends and changes.

We modify a portion of the matrix by changing certain links from 1 to 0 after predefining the matrix, which can be interpreted as adding noise. This introduces a degree of randomness and ambiguity, reducing the model's overfitting to the training data and enhancing its generalization ability, leading to better performance on unseen data. As the evolutionary process continues, we gradually and adaptively restore the elements that were initially set to 0, allowing the model to better accommodate changing traffic information. Our main contributions are summarized as follows:

- We design evolutionary graph neural networks for traffic prediction. The evolutionary process of the semantic adjacency matrix dynamically adjusts its structure based on the features and semantic relationships of the current data, iteratively evolving throughout the training cycle while preserving historical information. This design enhances the model's adaptability, enabling it to better handle complex and dynamic traffic data.
- We introduce an initial matrix adjustment by replacing approximately 50% of the values in the semantic neighborhood matrix with 0. This adds a degree of randomness and ambiguity, reducing the model's overfitting to the training data and improving its generalization ability, resulting in better performance on unseen data.
- Experiments are conducted on five real-world traffic flow datasets to demonstrate the superiority of the EGNODE method.

## Related work

Traffic congestion stands as a pervasive challenge in urban environments, exerting profound impacts on various facets of society. Not only does it result in substantial economic losses due to productivity declines and increased fuel consumption, but it also exacerbates environmental pollution, contributing to greenhouse gas emissions and air quality deterioration. Moreover, the adverse effects of traffic congestion extend beyond economic and environmental realms, significantly impinging upon the quality of life for urban residents. Long commute times, heightened stress levels, and diminished accessibility to essential services are among the myriad consequences that underscore the urgency of addressing congestion in urban transportation systems.

By forecasting traffic conditions with greater accuracy and foresight, policymakers and transportation authorities can devise more effective strategies for managing traffic flow, deploying resources efficiently, and implementing targeted interventions to mitigate congestion hotspots. Moreover, accurate traffic flow prediction facilitates the development of intelligent transportation systems (ITS) equipped with real-time monitoring and adaptive control mechanisms, thereby enhancing overall system resilience and responsiveness to dynamic traffic conditions. In the subsequent sections, we present a comprehensive overview of the extant literature on traffic flow prediction, delineating recent advancements in traditional methodologies grounded in time series analysis, machine learning techniques, and the burgeoning field of deep learning.

How to collect more accurate traffic information is an important prerequisite for solving the problem of traffic congestion, and the development of remote sensing technology has made a great contribution to the more accurate collection of traffic information. Remote sensing technology is used to collect information on the ground through a variety of sensors, UAVs and other devices. In the field of transport, remote sensing technology is widely used to collect traffic data, including the number of vehicles, vehicle speed, traffic flow, road congestion and so on. These data are essential for urban traffic management, road planning, and traffic

flow optimization. In terms of accuracy, traffic data collected by remote sensing technologies are usually highly accurate. This is because they can monitor the movement of vehicles on the road in real time and in some cases are able to identify the vehicle type and direction of travel.

However, the accuracy may be affected by a number of factors, such as weather conditions, equipment malfunctions, and data processing. Therefore, these factors need to be taken into account when using traffic data collected by remote sensing techniques, and data calibration and validation may be required to ensure the accuracy and reliability of the data. The development of remote sensing technology in the transport sector is also advancing[13,14]. With advances in sensor and UAVs technology, the performance and accuracy of remote sensing equipment continue to improve. Meanwhile, the application of technologies such as artificial intelligence and machine learning has made the processing and analysis of traffic data more efficient and precise. In addition, integration with other information technologies, such as cloud computing, the Internet of Things and big data analysis, further expands the scope of application of remote sensing technology in the field of transport[15,16], providing more possibilities for urban transport management and planning.

In the domain of traffic prediction, the academic community has traditionally relied on a range of conventional statistical methods and machine learning methods. Noteworthy among these are: (i) the K-Nearest Neighbor (KNN[8]) paradigm elucidated by Zhang et al. (2013), which predicts the traffic flow at a given node by evaluating the traffic behavior of its k nearest topological neighbors; (ii) the ARIMA distribution structure described by van den[17] which integrates autoregressive and moving average mechanisms for traffic flow prediction; (iii) the KNN paradigm elaborated by van den Berg et al. (1996), which predicts the vehicle flow at a specific node by assessing the traffic behavior of its k nearest topological neighbors; (iv) the KNN paradigm described by van den Berg et al. (1996), which integrates autoregressive and moving average mechanisms for traffic flow prediction; (v) the Seasonal Autoregressive Integrated Moving Average (SARIMA[18]) technique detailed by Williams et al. (2003), which extends the prediction capabilities of ARIMA through an algorithm adapted to periodic changes. However, these machine learning frameworks, besides being constrained by manual feature derivation, also demonstrate a tendency towards time dependency while significantly overlooking spatial interconnectedness, thereby limiting their potential to autonomously extract predictive features relevant to the field of traffic prediction.

With the emergence of deep learning paradigms, there has been a notable surge in academic interest, primarily attributed to their inherent capability in delineating intricate spatiotemporal interdependencies central to data-centric challenges. Pioneering deep learning structural frameworks integrate spatial and temporal dimensions, including Fully Connected Long Short-Term Memory (FC-LSTM[19]), Convolutional Long Short-Term Memory (ConvLSTM[20]), Spatiotemporal Residual Network (ST-ResNet[21]), Spatiotemporal Long Short-Term Memory (ST-LSTM[22]), and Spatiotemporal Convolutional Neural Networks (STCNN). These prototypes primarily employ convolutional and recurrent neural networks for analysis and inference, with their structures primarily designed for analyzing grid-like data topology, inadvertently disregarding the typical graph-like properties of traffic systems. In light of this, contemporary research has shifted towards utilizing graph-centric deep learning structures, particularly Graph Neural Networks (GNNs[23]), aimed at more intricately and comprehensively encapsulating spatial relationships existing within transportation infrastructure networks. For instance, a comprehensive review by Jiang[24] explores the application of GNNs in traffic forecasting, highlighting their effectiveness in capturing complex spatial interactions between different road segments and nodes within a traffic network. This review discusses various GNN-based models, including Spatio-Temporal Graph Convolutional Networks[9], which integrate temporal convolutions to simultaneously model spatial and temporal dependencies. The review underscores the potential of GNNs to enhance traffic prediction accuracy by exploiting the structured nature of traffic data and improving the handling of dynamic traffic patterns.

For instance, STGCN[9] is an exemplar model that significantly enhances traffic speed prediction by combining spatial and temporal information within a graph representation. Similarly, ASTGCN[12] utilizes temporal and spatial attention mechanisms to gather critical information from data. DCRNN[25] combines graph convolution with recurrent neural networks in an encoder-decoder manner. STG2Seq[26] employs a multiple gated graph convolutional module and a seq2seq architecture with attention mechanisms to make multi-step predictions. DSANet[27] is a correlated time series prediction system that uses CNN networks and a self-attention mechanism for spatial correlations. DHGNN[28] leverages a hybrid graph-based neural network for data-driven traffic incident prediction. LRGCN[29] introduces a framework for handling time-evolving graphs where the graph's structure changes over time. SLTTCN[30] leverages spatial linear transformers and a bidirectional temporal convolution network to predict future traffic flow by addressing dynamic spatial correlations and temporal dependencies. LSTSC[31] captures long-short term sequence correlations using attention mechanisms in the proposed multi-modal attention neural network, enhancing the accuracy and reliability of traffic flow predictions. ASTGNN[32] is able to effectively capture the dynamic changes, periodicity and spatial heterogeneity of traffic data in both temporal and spatial dimensions by designing new self-attention mechanisms and dynamic graph convolution modules to improve the prediction accuracy. Liu et al.[33] proposed a multiscale spatio-temporal attention-based mechanism to capture actions, exemplifying the wide range of applications of spatio-temporal attention mechanisms. LFA-ConvLSTM is a model based on the spatio-temporal unit of interpretable local flow attention mechanism (LFA) for traffic flow prediction. The method explains the relationship between different traffic flows well. LFA-ConvLSTM[34] combines the ConvLSTM module to learn traffic-specific features, the LFA module to model the correlation between traffic flows, and the feature aggregation module to merge the above two, which improves the prediction results well. Lastly, Z-GCNETs[35] propose a new time-aware zigzag topological-based spatio-temporal model. However, these methods often handle temporal and spatial information in isolation, disregarding the interdependent relationship between these two aspects of the data. To address this shortfall, STSGCN[36] introduces a novel approach, concurrently constructing models in both the temporal and spatial domains, thus unifying spatial and temporal components for a more comprehensive analysis and prediction. The emergence of graph-based models[37,38] heralds a new era in traffic flow prediction, offering the ability to capture

intricate spatiotemporal dependencies in real-world traffic data. This shift enables more accurate predictions, thereby bolstering traffic management, optimizing transportation systems, and ultimately enhancing people's travel experiences.

In summary, graph-based models such as STGCN, ASTGCN, and STSGCN demonstrate the ability to capture complex spatio-temporal dependencies in real-world traffic data, offering a transformative approach to traffic flow prediction. These models hold promise for significantly enhancing traffic management, optimizing traffic systems, and improving the overall travel experience. Nevertheless, these models are not immune to the common issue of oversmoothing encountered in GNN models, which limits their ability to build deeper models and extract more profound spatio-temporal representations. While traditional neural networks tend to perform better with increased depth due to their capacity to learn intricate feature representations, graph neural networks (GNNs) often do not experience the same benefit. Research indicates that GNN performance peaks with just two layers, and additional layers can actually degrade performance[39]. The primary reason for this is oversmoothing, where increasing the number of layers causes node representations to converge to similar values, thereby impairing the model's ability to capture rich spatial features. This issue severely restricts the depth of GNNs, making it challenging to capture deeper feature representations. In the context of traffic flow prediction, GNN models are similarly affected, and there is a notable lack of studies addressing the importance of network depth for capturing long-term dependencies in spatio-temporal forecasting.

Ordinary Differential Equations (ODEs) have seen growing interest in their application to machine learning, particularly for modeling continuous-time dynamics. A comprehensive survey by Rackauckas[40] provides an in-depth overview of ODEs and their integration with numerical methods and machine learning. This review covers the theoretical foundations of ODEs, their numerical solutions, and their application in various fields, including machine learning. It highlights the development of Neural Ordinary Differential Equations (NODEs) as a novel approach for continuous-time modeling, emphasizing their advantages in handling irregular time-series data and providing efficient memory usage. The survey also discusses various extensions and applications of ODEs in deep learning frameworks, illustrating their versatility and impact on modern machine learning techniques. Neural Ordinary Differential Equations (NODEs[10]) represents the forefront paradigm of deep learning, proposed by Ricky T. Q. Chen in a seminal research paper in 2018. Traditional neural networks employ fixed layers and parameters throughout the entire training process, whereas neural ODEs differ in that they adopt a continuous-time perspective, modeling data as a dynamic process evolving over time, thus fundamentally altering this approach. The fundamental concept of neural ODEs is to view the depth of the model as a continuous time process. The depth of neural ODE models is not strictly defined by pre-determined layers but dynamically evolves by integrating ordinary differential equations (ODEs[41]) that change over time. This adaptive adjustment of model complexity enables neural ODEs to flexibly capture the inherent complexity of the data. This approach addresses the oversmoothing issue frequently encountered in traditional GNN models, thereby enabling the construction of deeper network models and extraction of more information. NODEs have been applied in various research domains, including graph neural networks and traffic flow prediction. Specifically, as the only traffic prediction method based on ODEs, Spatial-Temporal Graph Ordinary Differential Equations (STGODE[11]) effectively leverages the advantages of ODEs, enabling the construction of deeper models to extract richer information. We introduced its shortcomings in the previous section, the semantic adjacency matrix generated at the initial stage is not well adapted to the complexity and variability of traffic data.

Recent approaches have increasingly employed neural differential equations (NDEs) to process traffic information. For example, STG-NCDE[42] combines graph neural networks (GNN) and neural controlled differential equations (NCDE) for traffic prediction. This method excels at handling complex time-series data with external control inputs. However, it has limitations. Compared to our model, STG-NCDE trains temporal and spatial data separately and does not adequately address the rich semantic information in traffic data, resulting in potentially suboptimal predictions when semantic content is high. MTGODE[43] combines dynamic graph neural networks and ordinary differential equations (ODEs). The model addresses the discontinuities, high complexity and dependence on predefined graph structures of discrete neural structures through continuous time aggregation and continuous graph propagation mechanisms. However, when MTGODE solves the problem of dependence on predefined graph structures based on continuous time aggregation and continuous graph propagation mechanism, it ignores the problem of dealing with the complex and changing semantic information of traffic information, which changes with time and other factors, which is overcome by our model. In contrast, GRAM-ODE[44] utilizes the same neural ordinary differential equation (NODE) framework as our model. It incorporates multiple interconnected ODE-GNN modules to capture diverse views of complex local and global spatio-temporal dependencies, leading to improved representations. However, GRAM-ODE also lacks in leveraging the semantic information within traffic data. Our model, featuring the Envolutional Graph Neural Network, effectively exploits the rich semantic information inherent in traffic data, providing a more comprehensive solution for traffic prediction.

In the field of spatio-temporal prediction, the spatio-temporal generative graph sparse coding method is widely applied. Its primary function is to extract representative spatio-temporal features from dynamic traffic data, thereby providing more accurate information support for traffic flow prediction. The ST-SGCCaps[45] method captures complex patterns within spatio-temporal graphs, effectively extracting key features that represent changes in the traffic system. These features include dynamic relationships of traffic flow variations over time and space. By leveraging sparse coding techniques, ST-SGCCaps compresses high-dimensional traffic data into low-dimensional sparse representations, thereby reducing computational complexity and enhancing model generalization capabilities. DTIGNN[46]models network-level traffic flow as a dynamic graph influenced by traffic signals and predicts future traffic flow data from generated spatio-temporal sparse graphs, effectively handling spatio-temporal traffic data. In summary, using spatio-temporal generative graph sparse coding to

process traffic information effectively extracts important spatio-temporal features from the data, and our model, employing an evolving network, better adapts to changes in traffic information.

We propose an evolutional graph neural network for traffic prediction, which is able to update the semantic adjacency matrix according to the iteration of the training cycle, and the evolution process of the semantic adjacency matrix can dynamically adjust the structure of the matrix according to the features and semantic relations of the current data, while retaining the historical information of the data, which improves the ability of the model to adapt to the complex and variable traffic data. We also designed the initial matrix adjustment, which simply means replacing about 50% of the values in the semantic adjacency matrix with 0. This introduces a certain degree of randomness and ambiguity, which can reduce the degree of overfitting of the model to the training data and improve the generalization ability of the model, thus obtaining better performance on unknown data.

## Preliminaries

In this section, we present the definition of the traffic prediction problem and the construction of two types of adjacency matrices.

### Traffic flow forecast

We define the road network as a graph $\mathcal{G} = (V, E, A)$, where $V$ denotes a collection of $N$ nodes, $E$ represents edges, and $A \in \mathbb{R}^{N\times N}$ signifies the adjacency matrix. The spatial characteristics of the traffic network are delineated by the topological arrangement of the graph $\mathcal{G}$, while temporal details are encoded in the feature attributes of the nodes. At any given time $t$, each node $i$ is associated with an observation vector $\mathbf{x}_t^i \in \mathbb{R}^F$, where $F$ denotes the length of the observation vector. The observations across all nodes at time $t$ are denoted as $X_t = \left(\mathbf{x}_t^1, \mathbf{x}_t^2, \ldots, \mathbf{x}_t^N\right) \in \mathbb{R}^{N\times F}$. The collective observations of all nodes over all times are represented as $\mathcal{X} = (X_1, X_2, \ldots, X_T) \in \mathbb{R}^{T\times N\times F}$. The primary aim in predicting traffic flow is to forecast future traffic conditions based on historical data within a defined temporal context, encapsulated by the input tensor $X$. This predictive task is achieved by defining a mapping function denoted as $f$. By analyzing past traffic data rigorously, the function $f$ extracts implicit patterns and tendencies, thereby providing predictive insights into the potential trajectory of traffic flow. Through the development of analytical models that capture the complex dynamics of the traffic network $\mathcal{G}$, the rich information contained within $X$ is leveraged to facilitate the training of the mapping function $f$. This enables the model to generate accurate and reliable predictions of future traffic scenarios, thus enhancing our capacity to manage and optimize traffic flow. Given the tensor $\mathcal{X}$ recorded within a transportation network $\mathcal{G}$, the objective of traffic forecasting is to establish a mapping function $f$ that utilizes the historical $T$ observations to forecast future traffic patterns for a subsequent duration $T'$.

$$[X_{t-T+1}, X_{t-T+2}, \ldots, X_t; \mathcal{G}] \xrightarrow{f} [X_{t+1}, X_{t+2}, \ldots, X_{t+T'}] \tag{1}$$

### Adjacency matrix construction

In the realm of traffic flow prediction, the adjacency matrix assumes paramount importance as a foundational concept, facilitating the representation of interconnections among discrete nodes within a road network. The spatial relationships between nodes can be succinctly encapsulated through the construction of an adjacency matrix, furnishing input features for models aimed at forecasting traffic flow. The process of crafting the adjacency matrix for a road graph entails the computation of distances between stations within the traffic network. Building upon the framework introduced by the Space-Time Graph Convolutional Network[9], the formulation of the weighted adjacency matrix, denoted as $W$, is articulated as follows:

$$A_{ij}^{sp} = \begin{cases} \exp\left(-\dfrac{d_{ij}^2}{\sigma^2}\right), & i \neq j \text{ and } \exp\left(-\dfrac{d_{ij}^2}{\sigma^2}\right) \geq \epsilon \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

Here, $d_{ij}$ represents the edge weight, determined by $D_{ij}$. The parameters $\sigma^2$ and $\epsilon$ serve as thresholds for regulating the dispersion and sparsity of the matrix $D$.

Moreover, the contextual similarity between nodes harbors valuable information that warrants consideration. To capture semantic relatedness, Dynamic Time Warping (DTW[47]) is employed, leading to the definition of a semantic adjacency matrix as follows:

$$A_{ij}^{se} = \begin{cases} 1, \text{ if } DTW(X^i, X^j) < \epsilon \\ 0, \text{ otherwise} \end{cases} \tag{3}$$

Here, $X^i$ denotes the time series of node $i$, and $\epsilon$ governs the sparsity of the adjacency matrix.

In the proposed method, two neighborhood matrices will be trained. After the initial use of the adjacency matrices, they will be preprocessed for normalization. Consider an adjacency matrix $A \in \mathbb{R}^{N\times N}$; a common procedure is to normalize it to $\tilde{A} = D^{-1/2}AD^{-1/2}$, where $D$ denotes the degree matrix corresponding to $A$. The matrix $\tilde{A}$ facilitates eigenvalue decomposition, where the eigenvalues are encapsulated in the range $[-1, 1]$. During the training phase, negative eigenvalues may lead to instability, and in order to address this issue, the introduction of self-loops is a well-accepted mitigation technique. In this study, a modified version of $\tilde{A}$ described in the literature is used:

$$\hat{\mathbf{A}} = \frac{\alpha}{2}(I + D^{-1/2}AD^{-1/2}) \tag{4}$$

where the hyperparameter $\alpha$ is restricted to the open interval (0, 1), ensuring that the eigenvalues of $\hat{A}$ are dispersed within the closed interval $[0, \alpha]$.

## Neural ODEs

We employ the spatio-temporal tensor as an input to enable concurrent processing of spatial and temporal information. The intricate spatio-temporal correlations are interwoven through tensor multiplication on each mode. To tackle the issue of oversmoothing, STGODE[11] introduces a restart distribution $H_0$, manifested as:

$$\mathbf{H}_{l+1} = \mathbf{H}_l \times_1 \hat{A} \times_2 U \times_3 W + \mathbf{H}_0 \tag{5}$$

Here, $\mathbf{H}_l \in \mathbb{R}^{N \times T \times F}$ denotes a spatial-temporal tensor capturing hidden embeddings of nodes in the $l$-th layer. The $\times_i$ operation signifies tensor-matrix multiplication on mode $i$. $\hat{A}$ represents the regularized adjacency matrix, $U$ is the temporal transformation matrix, $W$ is the feature transformation matrix, and $\mathbf{H}_0$ refers to the initial input of the Graph Neural Network (GNN), obtainable through an alternative neural network.

Subsequently, we express the above equation as:

$$\mathbf{H}_l = \sum_{i=0}^{l} \left( \mathbf{H}_0 \times_1 \hat{A}^i \times_2 U^i \times_3 W^i \right) \tag{6}$$

where the output representation $\mathbf{H}_l$ aggregates information from all layers, signifying that the final outputs collect information from no more than $l$-order neighbors without losing initial features.

However, the preceding equation incorporates an excessive number of parameters, rendering training challenging. Consequently, we substitute the discrete variable $n$ with a continuous variable $t$ and interpret the equation as a Riemann sum ranging from 0 to $n$, articulated as:

$$\begin{aligned} \mathbf{H}_n &= \sum_{i=0}^{n} \left( \mathbf{H}_0 \times_1 \hat{A}^i \times_2 U^i \times_3 W^i \right) \\ &= \sum_{i=1}^{n+1} \left( \mathbf{H}_0 \times_1 \hat{A}^{(i-1)\times\Delta t} \times_2 U^{(i-1)\times\Delta t} \times_3 W^{(i-1)\times\Delta t} \Delta t \right) \end{aligned} \tag{7}$$

where $\Delta t = \frac{t+1}{n+1}$ with $t = n$. As $n$ tends to $\infty$, we formulate the following integral:

$$\mathbf{H}(t) = \int_0^{t+1} \mathbf{H}_0 \times_1 \hat{A}^\tau \times_2 U^\tau \times_3 W^\tau \mathrm{d}\tau \tag{8}$$

Derivation of the above integral:

$$\frac{\mathrm{d}\mathbf{H}(t)}{\mathrm{d}t} = \mathbf{H}_0 \times_1 \hat{A}^{t+1} \times_2 U^{t+1} \times_3 W^{t+1} \tag{9}$$

However, $\hat{A}^{t+1}, U^{t+1}, W^{t+1}$ is challenging to compute, leading to the final ordinary differential equation obtained after another derivation and approximate simplification:

$$\frac{\mathrm{d}\mathbf{H}(t)}{\mathrm{d}t} = \mathbf{H}(t) \times_1 (\hat{A} - I) + \mathbf{H}(t) \times_2 (U - I) + \mathbf{H}(t) \times_3 (W - I) + \mathbf{H}_0 \tag{10}$$

The first-order Taylor expansion of the logarithmic operation, $\ln M \approx M - I$, further simplifies Eq.

Finally, the graphical feature representation mixed with temporal dynamics can be solved by putting the ordinary differential equations into the ODE solver (our model is solved using Euler's method):

$$\mathbf{H}(t) = \mathrm{ODESolve}\left( \frac{\mathrm{d}\mathbf{H}(t)}{\mathrm{d}t}, \mathbf{H}_0, t \right) \tag{11}$$

## Temporal convolution network

The TCN[48] model is constructed based on the architecture of Convolutional Neural Networks (CNNs), specializing in the analysis of time series data. In contrast to traditional Recurrent Neural Networks (RNNs) and Long Short-Term Memory Networks (LSTMs), TCN demonstrates proficiency in capturing extensive temporal dependencies and enjoys a significant edge in parallel computation. This computational advantage positions TCN as a superior solution for managing large-scale time series datasets.

At the heart of TCN lies the concept of Dilated Convolutions, which enhances the receptive field of convolutional kernels, facilitating the network's ability to comprehend information across different temporal scales. The dilated convolution operation is mathematically represented as follows:

$$\mathbf{y}(t) = \sum_{i=1}^{k} w(i) \cdot x(t - d \cdot (i-1)) \tag{12}$$

Here, $y(t)$ denotes the $t$th element in the output sequence, $x(t)$ represents the $t$th element in the input sequence, $w(i)$ indicates the weights of the convolutional kernel, $d$ denotes the dilation rate, and $k$ signifies the kernel size. By appropriately adjusting the dilation rate and kernel size, TCN effectively captures information across a spectrum of time intervals, thus enabling accurate modeling of time series data.

## Methods

In this section we describe how the data was collected, the use of the data and an introduction to our methodology.

### Traffic flow prediction in UAV-based urban traffic monitoring system

In the previous section, we introduced the process of traffic information collection, involving the deployment of sensors on roads for a specified duration and utilizing UAVs to collect data from these sensors. The collected data enables UAVs to construct spatio-temporal information maps of traffic, which are rich in detail and crucial for subsequent operations such as traffic prediction. The data collection process is illustrated in Fig. 1. However, before this information can be used for traffic prediction, further processing is necessary. In addition to spatio-temporal information, the data contains rich semantic details such as traffic accidents and emergencies, which significantly influence traffic flow prediction. Therefore, after data collection by UAVs, additional processing is required to construct spatial adjacency matrices and semantic adjacency matrices for accurate traffic flow prediction. Given the complexity and variability of traffic data, the construction of semantic adjacency matrices is particularly pivotal in adapting to the dynamic nature of traffic conditions.

### Traffic flow prediction model architecture

The historical traffic dataset serves as the foundation for deriving subsequent traffic flow scenarios by incrementally processing time intervals, Our model is also based on historical traffic information to make predictions, the framework of the model can be referred to Fig. 2. Our model's traffic flow prediction process begins with the construction of spatial and semantic adjacency matrices using data from the historical traffic dataset. We introduce an innovative modification to the semantic adjacency matrix by initially setting 50% of the matrix links from 1 to 0 before training, progressively restoring them through an evolutionary recovery process as training cycles iterate. The input data is initially processed by the Temporal Convolutional Network (TCN) for temporal information processing. Subsequently, the data is fed into the Neural Ordinary Differential Equation (NODE) solver for spatio-temporal data processing, employing Euler's method for solving. Following the solution, temporal information undergoes further integration and processing through TCN. Finally, prediction results are obtained via the maximum pooling layer and Multilayer Perceptron (MLP).

### Initial matrix adjustment

In our study, approximately 50% of the 1s in the semantic adjacency matrix are transformed into 0s as follows: $A_{ij}$ denotes the element at the $i$-th row and $j$-th column of the semantic adjacency matrix $A$. $R_{ij}$ represents an element at the same position as $A_{ij}$ in a random tensor with the same shape as $A$, where $R_{ij}$ is randomly sampled from a uniform distribution $[0, 1)$. $Z$ denotes a zero tensor with the same shape as $A$. Our operation is formulated as follows:

$$A_{ij} = \begin{cases} 0 & \text{if } R_{ij} < 0.5 \\ A_{ij} & \text{otherwise} \end{cases} \tag{13}$$

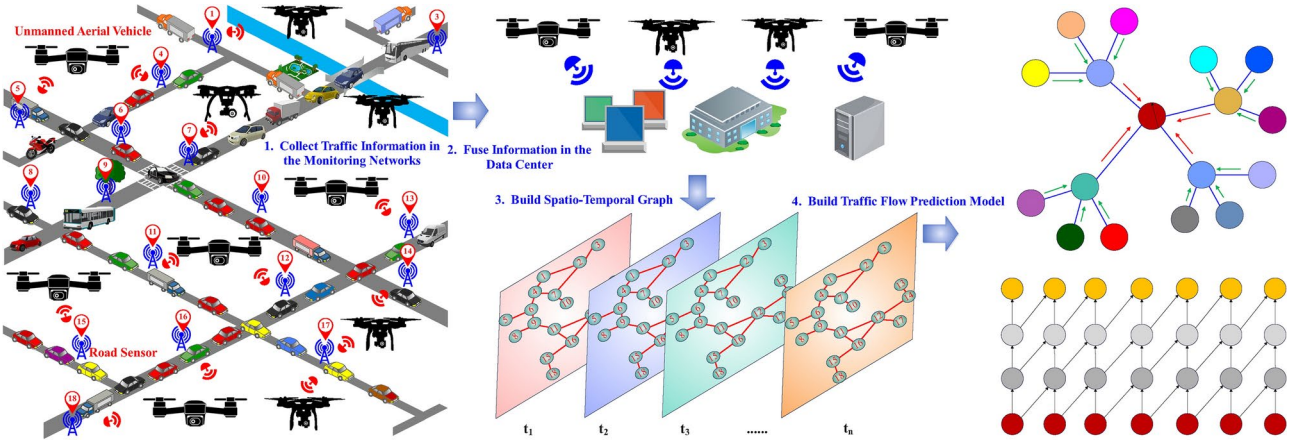



**Fig. 1**. Traffic flow prediction in UAV-based urban traffic monitoring system.

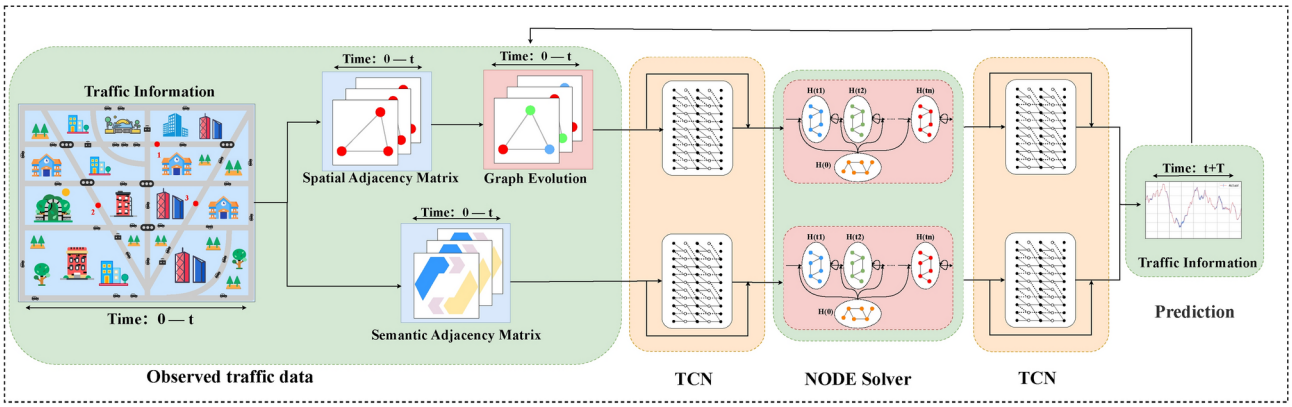



**Fig. 2**. The framework of EG-NODE.

This operation modifies each element $A_{ij}$ in the semantic adjacency matrix $A$. If the corresponding random number $R_{ij}$ is less than 0.5, $A_{ij}$ is replaced with 0; otherwise, it remains unchanged.

## Evolutional graph neural network

For a traffic graph $G = (V, E)$, where $V$ represents the set of nodes and $E$ represents the set of edges. Each node $v_i \in V$ has a feature vector $x_i$, representing the feature information of that node.

The main idea of extracting embeddings is based on the GraphSAGE model, which updates node representations by aggregating the information from neighboring nodes. Specifically, for node $v_i$, the GraphSAGE model aggregates the feature vectors of its neighboring nodes, then concatenates the aggregated result with the node's own feature vector, and finally updates the node representation through a neural network. The updating process can be divided into the following steps: first, aggregate the neighbor information. For each node $v_i$, aggregate the feature vectors $x_j$ of its neighbor nodes $v_j$ to obtain the aggregated vector $h_{\text{neigh}}$. Typically, mean pooling is used for aggregation, which computes the average of the feature vectors of all neighboring nodes.

$$h_{\text{neigh}} = \text{MeanPooling}(\{x_j | v_j \in \text{Ne}(v_i)\}) \tag{14}$$

Here, $\text{Ne}(v_i)$ represents the set of neighbor nodes of node $v_i$. Mean Pooling is a method of pooling vectors, which calculates the average of all elements in the vector. Specifically, the calculation formula for the aggregated vector $h_{\text{neigh}}$ is as follows:

$$h_{\text{neigh}} = \frac{1}{|\text{Ne}(v_i)|} \sum_{v_j \in \text{Ne}(v_i)} x_j \tag{15}$$

Here, $\text{Ne}(v_i)$ represents the number of neighbor nodes of node $v_i$, and $\text{Ne}(v_i)$ represents the set of neighbor nodes of node $v_i$.

After aggregating the neighbor information, update the node representation. Concatenate the node's own feature vector $x_i$ with the aggregated neighbor feature vector $h_{\text{neigh}}$ to obtain the new representation $h_i$ of node $v_i$.

$$h_i = [\text{Concat}(x_i, h_{\text{neigh}})] \tag{16}$$

Here, $Concat$ represents the concatenation operation of vectors. Then we update the node representation through a neural network, transforming the new representation $h_i$ nonlinearly using a neural network $\text{MLP}$ to obtain the updated representation $h_i'$ of node $v_i$.

$$h_i' = \text{MLP}(h_i) \tag{17}$$

Here, $\text{MLP}$ represents a Multi-Layer Perceptron, usually composed of multiple fully connected layers and nonlinear activation functions.

In summary, the process of extracting semantic adjacency matrix embeddings can be represented as:

$$\text{emb}_i = \text{MLP}([\text{Concat}(x_i, \text{MeanPooling}(\{x_j | v_j \in \text{Ne}(v_i)\}))]) \tag{18}$$

where $x_i$ represents the feature vector of node $v_i$, $\text{MeanPooling}(\{x_j | v_j \in \text{Ne}(v_i)\})$ represents the mean pooling result of the neighboring feature vectors of node $v_i$, and $\text{MLP}$ represents the nonlinear transformation operation of the neural network.

After that, we generate a new semantic neighbourhood matrix based on embedding and first calculate the similarity matrix between the nodes:

$$A_{\text{sm}} = \text{emb}_i \times \text{emb}_i^{\top}$$

$A_{\text{sm}}$ represents the generated similarity matrix and $\text{emb}_i^{\top}$ represents the transpose of $\text{emb}_i$. Subsequently, a new semantic neighbourhood matrix is generated based on the threshold value:

$$\text{A}_{new} = \begin{cases} 1, & \text{if A}_{sm} > 0.6 \\ 0, & \text{otherwise} \end{cases} \tag{19}$$

We then take its intersection with the initial semantic neighbor matrix. The reason for this is that semantic neighborhood matrices usually represent relationships between nodes, and these relationships may change in each training cycle. By calculating the intersection with the initial semantic neighborhood matrix, some relatively stable information can be retained and merged into the semantic neighbourhood matrix of the current cycle to obtain a more accurate representation of the nodes. $A_0$ denotes the initial semantic neighborhood matrix. The operation of taking intersection can be defined as:

$$A_{\text{intersection}} = A_0 \cap A_{\text{new}} \tag{20}$$

After adding the fetched intersection $A_{\text{intersection}}$ with the semantic neighborhood matrix of the previous cycle ($A_{\text{last}}^{(t-1)}$), the final semantic neighborhood matrix is obtained, which is fed into the model for the prediction work. The same operation is performed in the next cycle, and this semantic neighborhood matrix is also added to the $A_{\text{intersection}}$ extracted in the next cycle to get the final semantic neighborhood matrix for the next cycle, and the summing operation can be defined like this:

$$A_{\text{last}} = A_{\text{last}}^{(t-1)} + A_{\text{intersection}} \tag{21}$$

After being normalized by the matrices presented in the previous section, they are input into the model for processing. The concrete realization process of the above method can be referred to Fig. 3. The semantic neighborhood matrix processed in each cycle is further processed with the following formula:

$$\frac{\mathbf{dH}(t)}{\mathbf{d}t} = \mathbf{H}(t) \times_1 (\hat{A_{last}} - I) + \mathbf{H}(t) \times_2 (U - I) + \mathbf{H}(t) \times_3 (W - I) + \mathbf{H}_0 \tag{22}$$

This can then be solved using ODESolve. We have chosen to use Euler's method for the solution of ordinary differential equations. Euler's method is a numerical method for solving differential equations, where the approximate solution of a differential equation is obtained step-by-step by reducing the time continuum to discrete steps and by linearly approximating the current solution at the time step according to the derivative function of the differential equation. The ODE solver based on Euler's method is expressed as follows:

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + \Delta t \cdot \mathbf{f}(\mathbf{Y}_n, t_n) \tag{23}$$

where $\mathbf{Y}_n$ is the solution vector at the current time step $t_n$, $\mathbf{Y}_{n+1}$ is the solution vector at the next time step $t_{n+1} = t_n + \Delta t$, $\Delta t$ is the time step size, and $\mathbf{f}(\mathbf{Y}_n, t_n)$ is the value of the right-hand side function of the differential equation at the current time step. We can consider $\mathbf{Y}_n$ as $\left[\mathbf{H}(t)\right]$, and $\mathbf{f}(\mathbf{Y}_n, t_n)$ as the right-hand side part of the differential equation:

$$\mathbf{f}(\mathbf{Y}_n, t_n) = \frac{d}{dt}\left[\mathbf{H}(t_n)\right] \tag{24}$$

Thus, the specific formula for the Euler method solver is:

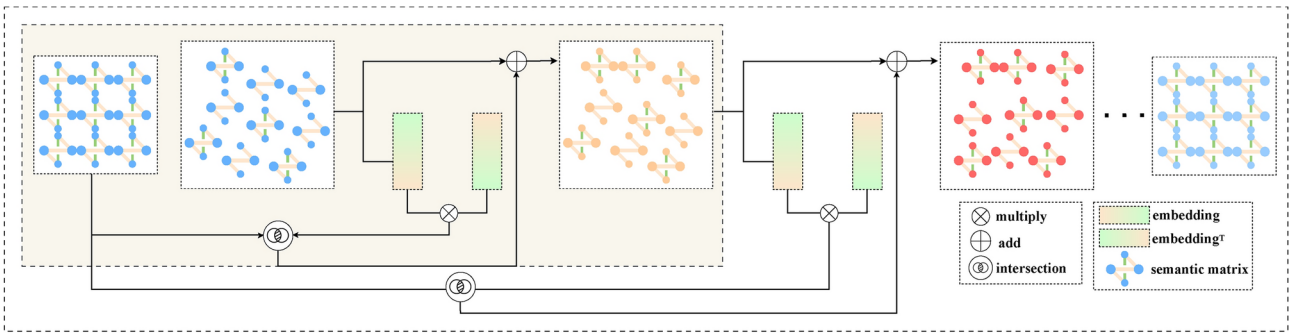



**Fig. 3**. The framework of graph evolution.

$$\left[\mathbf{H}(t)\right] = \left[\mathbf{H}(t)\right] + \Delta t_i \cdot \frac{d}{dt}\left[\mathbf{H}(t)\right] \tag{25}$$

After solving, we will then use TCN to further process the time information:

$$H_{out}^{l} = \begin{cases} X & , l = 0 \\ \sigma(W^{l} *_{d^l} H_{out}^{l-1}) & , l = 1, 2, \ldots, L \end{cases} \tag{26}$$

Here, $X \in \mathbb{R}^{N \times T \times F}$ represents the input of the TCN, $H_{out}^{l} \in \mathbb{R}^{N \times T \times F}$ denotes the output of the $l$-th layer of the TCN, and $W^{l}$ signifies the $l$-th convolutional kernel. To expand the receptive field and enable the network to capture broader temporal contexts, an exponential dilation rate of $d^{l} = 2^{l-1}$ is employed in the temporal convolution process. Additionally, the zero-padding strategy is employed to maintain the length of the time sequence unchanged throughout the convolutional operations.

After the second iteration of the Temporal Convolutional Network (TCN) layer, the input undergoes two subsequent stages to derive the ultimate output. Initially, the resultant feature map $\mathbf{H}_{\text{out}}$ undergoes information aggregation via a max-pooling layer. Max-pooling, serving as a downsampling technique, selects the maximum value within a local area, typically a rectangular window, of the feature map. This process reduces spatial dimensions while preserving essential information. Mathematically, the aggregation process is denoted as:

$$\mathbf{H}_{p} = \text{MaxPooling}(\mathbf{H}_{\text{out}}) \tag{27}$$

Here, $\mathbf{H}_{p}$ signifies the updated feature representation post the max-pooling operation.

Subsequently, the aggregated feature representation $\mathbf{H}_{p}$ is inputted into a multi-layer perceptron (MLP) for further nonlinear transformation. The MLP comprises several fully connected layers, each executing a linear transformation followed by a nonlinear activation function. The transformation within a typical MLP layer is expressed as:

$$\mathbf{H}^{(l+1)} = \sigma(W^{(l)}\mathbf{H}^{(l)} + b^{(l)}) \tag{28}$$

In this equation, $l$ denotes the index for the MLP layers, $\sigma$ represents the nonlinear activation function, and $W^{(l)}$ and $b^{(l)}$ symbolize the weight matrix and bias vector of the $l$-th layer, respectively. By iteratively applying such transformations, the final output is produced from the MLP's last layer.

Combining these processes, the model's prediction $\hat{Y}$ can be articulated as:

$$\hat{\mathbf{Y}} = \text{MLP}(\text{MaxPooling}(\mathbf{H}_{\text{out}})) \tag{29}$$

In this formulation, the function $\text{MaxPooling}()$ signifies the max-pooling operation conducted on the input $\mathbf{H}_{\text{out}}$, while the function $\text{MLP}$ denotes the subsequent nonlinear transformations executed by the multi-layer perceptron to yield the final prediction $\hat{Y}$.

## Model optimization

To mitigate the potential exacerbation of errors inherent in calibrating our computational framework, we strategically adopted the Huber loss paradigm to guide our error management approach. Diverging from the conventional squared error loss, the Huber loss function distinguishes itself with its balanced sensitivity to data points that deviate from the typical trend, often referred to as outliers. This attribute bestows resilience upon our mathematical model, equipping it with a poised equanimity when confronted with noisy data or phenomena that deviate from anticipated patterns. This methodology holds paramount significance in practical contexts, where datasets frequently harbor outliers capable of undermining the predictive accuracy of the model. Incorporating the Huber loss function establishes a harmonious equilibrium between the model's fidelity to the underlying data structure and its capacity to withstand the influence of aberrant data points. This fusion of fitting precision and robustness amplifies the reliability and accuracy of the model in real-world applications.

$$L(Y, \hat{Y}) = \begin{cases} \frac{1}{2}(Y - \hat{Y})^2 & , |Y - \hat{Y}| \leq \delta \\ \delta|Y - \hat{Y}| - \frac{1}{2}\delta^2 & , \text{otherwise} \end{cases} \tag{30}$$

In this equation, the symbol $\delta$ denotes a hyperparameter crucial for regulating the model's responsiveness to outliers.

## Algorithm implementation

Algorithm 1 delineates the structured methodology for conditioning the EG-NODE framework. This algorithmic blueprint furnishes a thorough roadmap for both the empirical application and conceptual comprehension of the underlying model dynamics. The EG-NODE algorithm embodies a predictive model that functions on a repertoire of inputs, encompassing training data $\mathbf{x}$, prospective data $\mathbf{Y}$, "Spatialfunc" for the construction of spatial adjacency matrices, "Semantfunc" for the construction of semantic adjacency matrices and "Evolution" for the evolution of semantic adjacency matrices. "TCN" stands for Temporal Convolution, and assorted hyperparameters, culminating in the derivation of a model endowed with optimized parameters $\theta$. We analyzed

the time complexity of the proposed method. The time complexities of the "Spatialfunc" and "Semantfunc" functions are $O(N^2)$ and $O(N^2 d)$, respectively, where $N$ represents the number of nodes, and $d$ denotes the dimensionality of the features. The 'Anodefunc' function has a time complexity of $O(N^2 + Nd)$.

**Input:** Training data $\mathbf{x}$, Future Data $\mathbf{Y}$, input length $t$, forecasting horizon $H$, batch size $B$, training epoch $E$, learning rate $\eta$.
**Output:** Forecast Model with parameter $\theta$.

**Function** Spatialfunc ($\mathbf{H}_{in}, \mathbf{A}^{\mathbf{sp}}$)
  $\widehat{\mathbf{A}} \leftarrow \text{adj_norm}(\mathbf{A}^{\mathbf{sp}})$
  **return** $\mathbf{H}_{\mathbf{n}}^{\mathbf{sp}}$

**Function** Semantfunc ($\mathbf{H}_{in}, \mathbf{A}^{\mathbf{se}}$)
  $\widehat{\mathbf{A}} \leftarrow \text{adj_norm}(\mathbf{A}^{\mathbf{se}})$
  $\mathbf{H}_n \leftarrow \widehat{\mathbf{A}}\mathbf{H}_{in} - \mathbf{H}_{in}$
  **return** $\mathbf{H}_{\mathbf{n}}^{\mathbf{se}}$

**for** *number of epochs until convergence* **do**
  **Function** Anodefunc ($\mathbf{H}_n$)
    $\mathbf{A}_{last} \leftarrow \text{Evolution}(\mathbf{A}^{\mathbf{se}})$
    $\widetilde{\mathbf{H}} \leftarrow \text{TCN}(\mathbf{H}_n)$
    $\mathbf{H}_t \leftarrow \text{ODESolve}(\widetilde{\mathbf{H}}, \mathbf{H}_0, t)$
    **return** $\mathbf{H}_t$
  $\widetilde{\mathbf{H}_{out}} \leftarrow \text{TCN}(\mathbf{H}_t)$
  $\mathbf{H}_{out}$ undergoes information aggregation via a max-pooling layer and is then output through an MLP.
  $\hat{Y} \leftarrow \text{MLP}(\text{MaxPooling}(\mathbf{H}))$
  $l \leftarrow \text{Huber Loss}(\hat{\mathscr{Y}}, \mathscr{Y})$
  $\theta \leftarrow \theta - \Delta_\theta l$

**Algorithm 1**. EG-NODE algorithm

## Experiments

In this section, we conduct comparative experiments on five real benchmark datasets to demonstrate the performance of our model. We perform ablation experiments on our model to demonstrate the effectiveness of our model innovations.

### Datasets

The PeMS[4] system is widely recognized as a comprehensive and practical resource for road traffic data, crucial for both prediction and analysis of traffic flow. This system includes vehicle counts and motorway occupancy data collected from thousands of locations, covering all lanes, ramps, and on-ramps.

Traffic data collection technologies used in PeMS include radar, laser, and magnetometers, among others. The data is compiled from multiple sensors distributed across various locations, capturing data at 5-min intervals over several days. Typically, in many studies, a portion of data from specific regions or time periods is extracted from the PeMS system and represented as "PeMSD+Number." For example, the PeMSD4 dataset consists of traffic flow data from 307 detectors, recorded at 5-min intervals over 59 days. Each detector collects data across three dimensions: flow rate, average speed, and average occupancy. Average occupancy, a key indicator of road utilization, is divided into temporal and spatial occupancy. Temporal occupancy measures the proportion of time a road segment is occupied during the observation period, while spatial occupancy calculates the ratio of the total road area occupied by vehicles to the total road area in the observed section.

Originating from the California Department of Transportation, the PeMS system encompasses comprehensive real-time traffic flow data from highways and urban thoroughfares. It is gathered through an extensive network of detectors placed on major highways in key Californian cities, ensuring thorough monitoring. This data provides researchers and engineers with rich, invaluable resources for refining advanced traffic flow prediction models and devising sophisticated traffic management strategies. Its extensive coverage and real-time nature make it ideal for analyzing traffic system dynamics and addressing related challenges.

The METR-LA dataset[49] is a publicly available dataset widely used in traffic prediction and intelligent transportation systems research. It holds significant importance, particularly in fields such as time-series data analysis and graph neural networks. This dataset provides detailed records of traffic flow information in the Los Angeles area of California, USA, including data from multiple traffic monitoring sensors. These sensors are distributed across most of the major roads in Los Angeles, continuously monitoring vehicle speed and traffic flow in real time. Collected at regular intervals, the METR-LA dataset offers comprehensive traffic information across different times and geographical locations, effectively reflecting the complex and dynamic traffic conditions of Los Angeles.

This study leverages six real-world traffic datasets-PeMSD3, PeMSD4, PeMSD7, PeMSD8, PeMS-BAY[50] and METR-LA-all of which are obtained from real-time traffic scenarios at regular intervals. Detailed information on these datasets is provided in Table 1. The primary goal is to validate the effectiveness of our model by comparing it with recent methodologies in traffic flow prediction. In this experimental setup, data is collected at 30-s intervals and aggregated into 5-min segments, resulting in 288 time steps per day. Below is a detailed description of each dataset:

- PeMSD3: A component of the PeMSD dataset, PeMSD3, comprises real-time traffic flow data sourced from the highways of California.
- PeMSD4: This subset features an augmented sensor network, providing researchers with a more granular depiction of traffic flow dynamics, thereby facilitating deeper analysis of traffic phenomena and patterns.
- PeMSD7: A variant within the PeMSD dataset, PeMSD7, focuses on traffic flow data specifically within the Los Angeles area of California. This subset may include data from urban roads, enabling researchers to scrutinize urban traffic characteristics and trends.
- PeMSD8: Extending the PeMSD dataset, PeMSD8 delivers an expanded volume of traffic data, potentially encompassing a broader geographical region and road network. This subset offers opportunities for a more comprehensive investigation into traffic flow prediction and the evaluation of algorithmic performance on a larger scale.
- PeMS-BAY: As a benchmark dataset used for traffic prediction, the PeMS-BAY dataset gathers traffic data from a sensor network in the Bay Area, spanning multiple highways. Each sensor captures key data, including vehicle speed, traffic flow, and occupancy.

## Baselines

In our empirical investigation, we subjected the proposed model to a thorough evaluation against a wide array of baseline models. This ensemble encompassed 8 distinct forecasting architectures outlined in the related work section of this manuscript. The comparative analysis extended beyond state-of-the-art methodologies, encompassing a diverse range of seminal and prototypical forecasting models to establish a comprehensive performance benchmark. After a meticulous and equitable assessment, the findings unequivocally validated the superior predictive capability of ADM-STNODE, reaffirming its emergent status as a benchmark among its contemporary counterparts.

- HA[51]: This method employs the mean value of the preceding 12 temporal segments to forecast the forthcoming value.
- ARIMA[5]: Auto-Regressive Integrated Moving Average model, which is a well-known statistical model of time series analysis.
- TCN[48]: This method comprises a sequence of causal convolutional layers characterized by progressively increasing dilation factors following an exponential pattern.
- STGCN[9]: This method utilizes graph convolution and 1D convolution to capture spatial dependencies and temporal correlations respectively.
- ASTGCN[12]: This method employs attention mechanisms to extract spatio-temporal information from traffic flow data for prediction purposes.
- STGODE[11]: This method is based on ODE's spatio-temporal.
- Z-GCNETs[35]: Thigraph neural networks for traffic prediction.s method introduces the concept of zigzag persistence in time-aware GCN.
- VAR[6]: The method of Vector Autoregression models the linear relationships between multiple time series by predicting each variable based on its own and others' past values.
- FC-LSTM[52]: This method is LSTM with a fully connected hidden unit.
- DCRNN[49]: This method combines graphical convolution and recurrent neural networks in a coder-decoder fashion.
- GraphWaveNet[53]: Graph WaveNet integrates adaptive graph convolution with dilated causal convolution to capture spatio-temporal dependencies.
- MSTGCN[54]: This method is another version of ASTGCN, which gets rid of the spatio-temporal attention.
- STG2Seq[26]: The method uses a multiple-gated graph convolutional module and a seq2seq architecture with attention mechanisms to make multi-step predictions.
- DSANet[55]: This method is a correlated time series prediction model using CNN networks and a self-attention mechanism for spatial correlations.

| Dataset | Time steps | Time range | Sample size | Nodes | Edges |
|---|---|---|---|---|---|
| PeMSD3 | 26,208 | 09/2018–11/2018 | 515520 | 358 | 547 |
| PeMSD4 | 16,992 | 01/2018–02/2018 | 1764480 | 307 | 340 |
| PeMSD7 | 28,224 | 05/2017–08/2017 | 8866560 | 883 | 866 |
| PeMSD8 | 17,856 | 07/2016–08/2016 | 1958400 | 170 | 295 |
| PeMS-BAY | 52,116 | 01/2017–05/2017 | 16848000 | 325 | 2369 |
| METR-LA | 34,272 | 03/2012–06/2012 | 6024320 | 207 | 1515 |

**Table 1**. The summary of the datasets used in our work.

- DHGNN[28]: The method involves data-driven traffic incident prediction using a hybrid graph-based neural network.
- LRGCN[29]:The method handles time-evolving graphs in which the graph's structure changes over time.
- ASTGNN[32]: This method utilizes leverages self-attention and dynamic graph convolution to achieve superior traffic forecasting performance.
- DTIGNN[46]: This approach uses dynamic graphics influenced by traffic signals to model network-level traffic flows from sparse data.
- GRAM-ODE[44]: A graph-based multi-ode neural network spatio-temporal traffic prediction model.
- LFA-ConvLSTM[34]: This method uses an interpretable local flow attention mechanism to improve traffic flow prediction, outperforming existing methods in accuracy and efficiency.

## Experimental evaluation criteria

In model evaluation, we integrate three key performance metrics to comprehensively assess model effectiveness and predictive accuracy. These metrics include Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE). These metrics are chosen due to their widespread use and reliability in evaluating predictive model performance.

- RMSE quantifies the square root of the average error between model predictions and actual observations, offering insight into the magnitude and distribution of prediction errors. It penalizes large error values, with higher RMSE values indicating poorer model performance.

$$\mathbf{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{31}$$

- MAE calculates the average absolute error of the model, representing the average of the absolute differences between model predictions and actual observations. It provides a measure of the average error magnitude across different observations and is not influenced by outliers, potentially making it more robust for datasets with extreme values.

$$\mathbf{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{32}$$

- MAPE, expressed in percentage terms, measures the percentage error of the model by computing the average of the ratios of the differences between model predictions and actual observations to the actual observations themselves. It assesses the relative accuracy of the model across different observations, aiding in understanding the model's precision relative to the actual values.

$$\mathbf{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100\% \tag{33}$$

## Experimental settings

The experimental procedures were carried out on a server running Windows, equipped with specific hardware components. The central processing unit (CPU) utilized was an AMD Ryzen 7 6800H, paired with Radeon Graphics to deliver significant computational capabilities. Additionally, the system featured a dedicated graphics processing unit (GPU), the NVIDIA®GeForce RTX 3060 laptop GPU, which supported efficient parallel processing and accelerated the computational tasks. Each network layer was designed with up to four blocks derived from the decisional constant micro-differential ADM-STNODE. The datasets were partitioned into training, validation, and test sets in a 6:2:2 ratio. Historical data spanning one hour was employed to forecast future traffic conditions. The hidden dimensions for the TCN blocks were configured as 64, 32, and 64, respectively. The regularization hyperparameter $\alpha$ was set to 0.8, with spatial adjacency matrix thresholds $\sigma$ and $\epsilon$ set to 10 and 0.5, respectively, and the semantic adjacency matrix threshold $\epsilon$ was set to 0.6. The model was trained using the Adam optimizer with a learning rate of 0.01. The network depth consists of six layers, and the model integrates two TCNs and one NODE, effectively forming two stacked combinations of these components. Both spatial and semantic adjacency matrices were provided as separate inputs for model training.

## Comparative experiment

We conducted an exhaustive analysis across seven prominent real-world traffic datasets: PeMSD3, PeMSD4, PeMSD7, PeMSD8 and METR-LA. Comparative experiments were meticulously executed on these datasets, as outlined in Tables 2 and 3. Notably, the EG-NODE consistently demonstrated superior performance across all considered datasets. For instance, on PeMSD3, it achieved a MAE of 16.41, RMSE of 26.86, and MAPE of

15.30, indicative of its remarkable predictive accuracy. Notably, it significantly outperformed traditional HA and ARIMA models, with the HA model yielding an MAE of 30.81, RMSE of 51.33, and MAPE of 32.78 on the same dataset.

Similarly, on the PeMSD4 dataset, the EG-NODE exhibited superior performance with an MAE of 19.25, RMSE of 31.37, and a MAPE of 12.71, surpassing the HA model which resulted in an MAE of 37.13, RMSE of 58.26, and a MAPE of 28.68. Likewise, on the PeMSD7 dataset, the EG-NODE showcased the lowest forecasting errors with an MAE of 21.69, RMSE of 35.08, and MAPE of 9.12.

In the comparative experiment on the METR-LA dataset, the EG-NODE model achieved an MAE of 3.63, demonstrating lower errors compared to other models such as LFA-ConvLSTM (3.65), DCRNN (3.62), and GRAM-ODE (3.69). Additionally, its MAPE was 10.19%, significantly lower than that of all other models.

A consistent observation across all datasets is the superior performance of advanced neural network-based models, particularly those incorporating EG-NODE architectures, compared to traditional statistical models such as HA and ARIMA. While models like STGODE and Z-GNETs also demonstrated commendable performance, the EG-NODE consistently yielded the best results, underscoring its efficacy as a reliable traffic forecasting methodology.

To validate the enhanced predictive accuracy of our designed model, we conducted a comprehensive assessment using the PeMSD4 dataset, juxtaposing the predicted outputs with empirical traffic data. Figure 4 illustrates a series of plots elucidating the comparative performance of the models, facilitating a clear understanding of their efficacy.

The depicted graphs vividly illustrate the superior alignment of our model's predicted trajectories with the empirical traffic data. This congruence underscores the nuanced accuracy of our model, substantiating its minimal deviation from real observations and thereby establishing its position as an unparalleled and precise forecasting tool.

Conversely, notable disparities emerge between the comparative predictions of the STGODE, ASTGCN, and STGCN models and the actual traffic indicators in the PeMSD4 dataset. These disparities are particularly pronounced under conditions of high vehicle densities and rapid traffic flow perturbations, indicative of the limited capacity of these models to generalize and predict intricate traffic dynamics. A meticulous comparative analysis unequivocally demonstrates the superior prediction accuracy of our model.

In conclusion, the experimental findings underscore the outstanding performance of the EG-NODE model in traffic prediction tasks. Its consistent ability to reduce forecasting error margins suggests its potential practical applicability across various intelligent transportation scenarios. Future research endeavors could further explore additional datasets and real-world traffic conditions to validate and potentially enhance the adaptability and effectiveness of the EG-NODE model for traffic forecasting.

## Hyparameters analysis

We have verified the performance of the model by conducting analytical experiments on two aspects of the model.

*Oversmoothing analysis*

In the field of traffic flow prediction, traditional graph neural network (GNN) architectures have achieved impressive results. Nevertheless, these models still face a critical impasse, especially at advanced stages of deep training. This dilemma mainly stems from the oversmoothing problem, i.e., the representations of the nodes tend to converge after successive layers of graph convolution, thus obliterating the uniqueness of each node and ultimately inhibiting the improvement of the prediction accuracy during the iterative training process.

The Spatio-Temporal Graph Ordinary Differential Equation (STGODE) methodology emerges as a novel solution to the oversmoothing bottleneck problem by seamlessly integrating the Noodular Ordinary Differential Equation (NODE) into the GNN framework. Based on the STGODE methodology, we introduce EG-NODE, which not only supports the NODE-centric paradigm, but also provides an innovative treatment of the traffic information adjacency matrix. We perform a comprehensive empirical test by using the PeMSD4 traffic dataset, it maintains robustness to supersmoothing irrespective of the network depth without sacrificing its predictive power.

The evaluation metrics we use to quantify the performance of different architectural configurations are MAE and RMSE. Figure 5 graphically depicts the performance of the model as a function of network depth. As an example of loss MAE on PeMSD4 dataset, as the depth of the model is gradually increased from 2 to 6, the loss of our model does not increase with the depth, but rather the effect is getting better and better, from 20.1 to 19.68. It is noteworthy that EG-NODE consistently maintains the representation of the major nodes and shows no signs of performance degradation with increasing layering. The experimental results clearly demonstrate that our EG-NODE model can easily cope with the ultra-smoothing challenge and completely outperforms traditional deep GNN frameworks in prediction tasks. The resilience demonstrated by the model in refining complex and persistent spatio-temporal traffic patterns solidifies its value from both empirical and theoretical perspectives.

To analyze the phenomenon of over-smoothing related to random initialization, we removed the random initialization setting from our model. This means that in the initial semantic adjacency matrix, we did not set 50% of the ones to zero. The experimental results are clearly illustrated in Fig. 5, where "no-RI" represents the absence of random initialization. The results indicate that the addition or removal of random initialization does not affect the over-smoothing phenomenon.

*Threshold analysis*

Introducing a threshold in the adjacency matrix significantly impacts the model's performance in several ways. By removing weaker connections, the adjacency matrix becomes sparser. This sparsity reduces the computational

| | PeMSD3 | | | PeMSD4 | | | PeMSD7 | | | PeMSD8 | | | PeMSD-BAY | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | MAE | RMSE | MAPE | MAE | RMSE | MAPE | MAE | RMSE | MAPE | MAE | RMSE | MAPE | MAE | RMSE | MAPE |
| HA | 30.81 | 51.34 | 32.80% | 37.13 | 59.26 | 28.68% | 44.36 | 64.34 | 23.51% | 33.78 | 58.24 | 26.89% | 5.83 | 8.31 | 10.09% |
| ARIMA | 34.41 | 46.68 | 32.94% | 32.85 | 47.94 | 23.67% | 37.96 | 58.84 | 19.21% | 31.27 | 44.34 | 22.75% | 3.39 | 6.52 | 8.31% |
| VAR | 23.75 | 38.10 | 23.80% | 24.23 | 38.55 | 17.24% | 49.82 | 65.14 | 31.29% | 19.01 | 27.06 | 13.04% | 3.18 | 5.65 | 7.25% |
| FC-LSTM | 21.33 | 35.13 | 23.30% | 26.75 | 40.65 | 18.23% | 29.98 | 45.96 | 13.25% | 23.09 | 35.17 | 14.99% | 3.30 | 5.65 | 7.48% |
| TCN | 19.31 | 33.42 | 19.86% | 23.34 | 37.27 | 15.59% | 32.72 | 42.23 | 14.26% | 22.72 | 35.79 | 14.03% | 3.07 | 4.98 | 5.60% |
| DCRNN | 17.99 | 30.31 | 18.34% | 21.22 | 33.44 | 14.17% | 25.22 | 38.61 | 11.82% | 16.82 | 26.36 | 10.92% | 2.09 | 4.76 | 4.91% |
| GraphWaveNet | 19.11 | 31.78 | 18.98% | 24.83 | 39.67 | 17.34% | 26.49 | 41.52 | 11.77% | 18.21 | 29.89 | 12.23% | 2.08 | 4.69 | 4.97% |
| MSTGCN | 19.56 | 31.93 | 23.86% | 23.96 | 37.21 | 16.33% | 29.00 | 43.73 | 14.30% | 19.00 | 29.15 | 12.38% | 2.58 | 4.935 | 5.28% |
| STG2Seq | 19.13 | 28.98 | 21.42% | 25.10 | 38.39 | 18.65% | 32.67 | 47.14 | 20.12% | 20.09 | 30.56 | 17.26% | 2.12 | 4.78 | 4.89% |
| DSANet | 21.29 | 34.56 | 23.21% | 22.78 | 35.77 | 16.03% | 31.26 | 49.11 | 14.45% | 17.14 | 26.99 | 11.32% | 3.16 | 4.98 | 5.82% |
| DHGNN | 16.99 | 28.15 | 17.02% | 20.96 | 32.66 | 14.55% | 22.73 | 35.68 | 10.28% | 18.10 | 28.56 | 10.82% | 2.35 | 4.82 | 4.56% |
| LRGCN | 17.96 | 30.36 | 18.52% | 20.04 | 33.28 | 17.43% | 22.53 | 36.26 | 10.35% | 16.41 | 26.37 | 11.89% | 2.17 | 4.86 | 4.61% |
| STGCN | 17.25 | 30.20 | 17.29% | 21.46 | 34.72 | 16.63% | 25.13 | 39.25 | 11.01% | 17.54 | 27.09 | 11.20% | 2.48 | 5.62 | 5.78% |
| ASTGCN | 17.54 | 29.59 | 17.22% | 22.84 | 34.84 | 15.99% | 24.11 | 36.89 | 10.83% | 18.55 | 28.16 | 11.66% | 2.12 | 4.86 | 4.60% |
| STGODE | 16.43 | 26.89 | 16.59% | 19.89 | 32.81 | 15.48% | 22.55 | 36.74 | 10.17% | 16.31 | 25.49 | 10.32% | 2.32 | 4.83 | 4.55% |
| Z-GCNETs | 16.75 | 28.18 | 16.39% | 19.65 | 31.62 | 12.79% | 21.77 | 35.17 | 9.26% | 15.76 | 25.18 | 10.05% | 2.06 | 4.77 | 4.63% |
| DTIGNN | 16.59 | 27.16 | 16.11% | 19.55 | 31.53 | 12.80% | 21.71 | 35.12 | 9.19% | 15.72 | 25.15 | 10.01% | 2.15 | 4.63 | 4.78% |
| ASTGNN | 16.60 | 27.13 | 16.12% | 19.49 | 31.56 | 12.83% | 21.75 | 35.14 | 9.20% | 15.70 | 25.16 | 10.06% | 2.05 | 4.89 | 4.61% |
| GRAM-ODE | 16.79 | 27.02 | 16.33% | 19.35 | 31.41 | 13.29% | 21.72 | 35.15 | 9.21% | 15.73 | 25.19 | 9.99% | 2.05 | 4.73 | 4.55% |
| LFA-ConvLSTM | 16.42 | 26.91 | **15.29%** | 19.26 | 31.38 | 12.72% | **21.66** | 35.11 | 9.15% | 15.71 | 25.14 | 9.99% | **2.01** | 4.72 | 4.51% |
| **EG-NODE** | **16.41** | **26.86** | 15.30% | **19.25** | **31.37** | **12.71%** | 21.69 | **35.08** | **9.12%** | **15.66** | **25.12** | **9.98%** | 2.03 | **4.70** | **4.49%** |

**Table 2**. Performance comparison on PeMSD3, PeMSD4, PeMSD7, PeMSD8, and PeMSD-BAY.

| Model | MAE | RMSE | MAPE |
|---|---|---|---|
| HA | 9.24 | 16.20 | 20.64% |
| ARIMA | 6.93 | 13.26 | 17.38% |
| GRU-ED | 5.85 | 11.33 | 15.42% |
| VAR | 5.96 | 11.43 | 15.32% |
| FC-LSTM | 5.66 | 10.32 | 15.46% |
| TCN | 5.00 | 9.99 | 12.30% |
| DCRNN | 3.62 | 7.59 | 10.53% |
| GraphWaveNet | 3.70 | 7.44 | 11.00% |
| MSTGCN | 4.96 | 10.15 | 12.00% |
| STG2Seq | 3.72 | 7.90 | 10.59% |
| DSANet | 5.23 | 10.01 | 12.53% |
| DHGNN | 3.74 | 7.46 | 10.28% |
| LRGCN | 4.36 | 8.87 | 12.31% |
| STGCN | 4.61 | 9.39 | 12.73% |
| ASTGCN | 4.36 | 8.83 | 12.34% |
| STGODE | 3.72 | 7.56 | 10.28% |
| Z-GCNETs | 3.70 | 7.58 | 10.26% |
| DTIGNN | 3.74 | 7.56 | 10.33% |
| ASTGNN | 3.71 | 7.59 | 11.01% |
| GRAM-ODE | 3.69 | 7.48 | 10.21% |
| LFA-ConvLSTM | 3.65 | **7.39** | 10.22% |
| EG-NODE | **3.63** | 7.44 | **10.19%** |

**Table 3**. Performance comparison on METR-LA. Significant values are in bold.

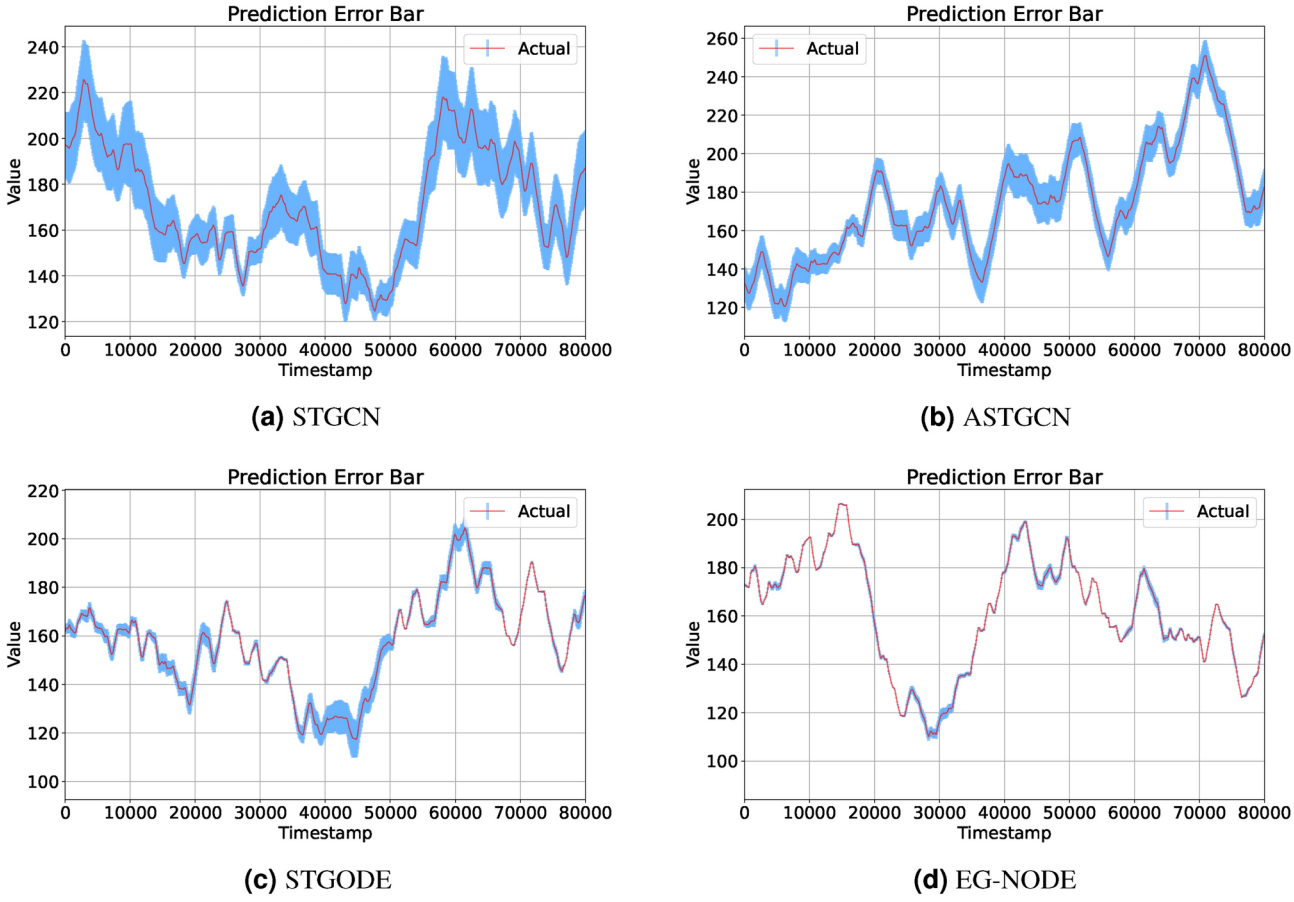



**Fig. 4**. Comparison of predictions error bars.

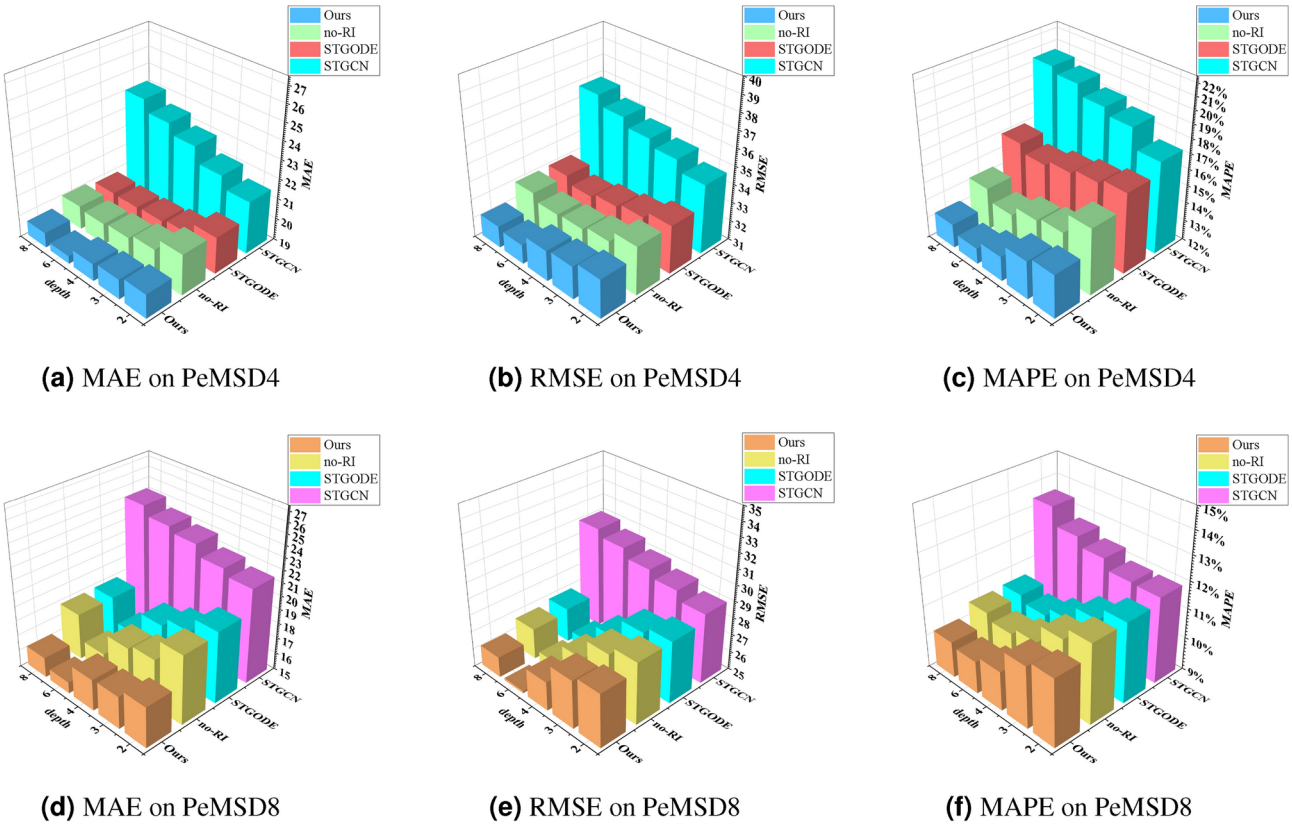



**Fig. 5**. The impact of network depth on the performance of MTEC-AODE on PeMSD4 and PeMSD8.

complexity of the model, as fewer connections need to be processed during training and inference. Consequently, this speeds up computation and lowers memory requirements, which is crucial for handling large-scale datasets such as PeMSD4. Real-world data often contains noise or irrelevant connections that can negatively affect model performance. By setting a threshold, these weaker and potentially noisy connections are filtered out, resulting in a clearer and more robust data representation.

To empirically analyze the impact of different threshold values on the performance of the semantic adjacency matrix constructed by our model, we conducted experiments to calculate the MAE, RMSE, and MAPE for various thresholds on the real datasets PeMSD4 and PeMSD3. The results are clear that in terms of the results on the dataset PeMSD4, the model achieves the best results at a threshold size of 0.6. As shown in Fig. 6, based on the data observation and analysis, it can be seen that at a threshold value of 0.6, the model achieves the best results under all three metrics. At this threshold, MAE is 19.25, RMSE is 31.37, and MAPE is 12.71. Therefore, the threshold value of 0.6 can be considered as the parameter setting that achieves the best results. This suggests that a well-chosen threshold can balance the trade-off between maintaining significant connections and removing noise, thereby optimizing the overall model performance.

*Loss analysis*

We trained the model on the PeMSD4 and PeMSD8 datasets and observed its convergence, as shown in Fig. 7. For the PeMSD4 dataset, we observed that the model converges more rapidly in the initial phase. In the pre-training phase, the loss function decreases rapidly and the model's fit to the training data improves significantly. As the training progresses, the decline of the loss function gradually slows down and eventually stabilises at a lower level. Compared with PeMSD4, the model converges slower on PeMSD8. In the initial stage, the loss function decreases at a faster rate, but this rate gradually slows down and eventually stabilises as training progresses. Nevertheless, the model still achieves satisfactory convergence on PeMSD8. Overall, our model shows good convergence performance on different datasets.

*Sample complexity analysis*

To thoroughly evaluate the sample complexity of our proposed method, we designed a series of experiments aimed at observing the model's performance under varying sample sizes. Specifically, we randomly selected data covering different continuous time spans-three, five, and seven days-within the PeMSD4 and PeMSD8 datasets. By using these datasets of different scales, we were able to assess the impact of sample size on the model's performance. In our experiments, we employed multiple evaluation metrics to comprehensively measure the model's performance, including MAE, MAPE, and RMSE. These metrics capture different aspects of prediction accuracy, reflecting the absolute error, relative error percentage, and the average squared error, respectively.

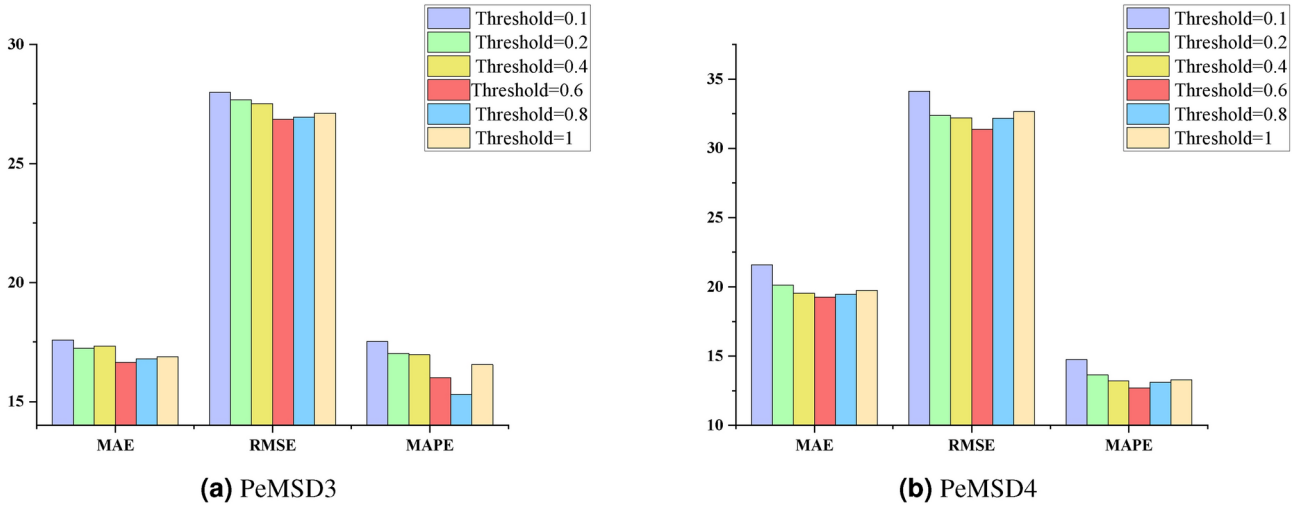



**Fig. 6**. Threshold analysis on PeMSD3 and PeMSD4.

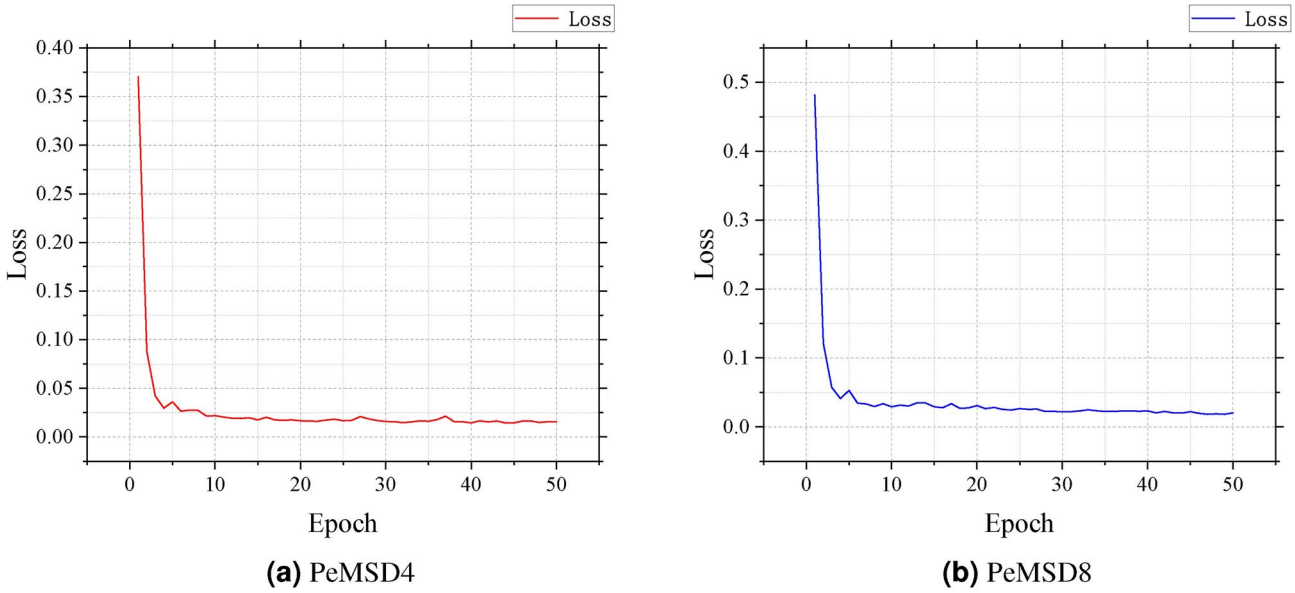



**Fig. 7**. Loss analysis.

The experimental results are shown in Fig. 8. The experimental results show a significant improvement in model performance as the data volume increases. Specifically, when the sample size is smaller, the model exhibits relatively high errors, likely due to the insufficient data to fully capture the complexity of the dataset, which limits the model's generalization capability. However, as the sample size increases to five or seven days, the model's error decreases markedly, indicating a better understanding of data trends and patterns, leading to improved predictive accuracy. Further analysis reveals that data spanning longer periods not only includes more samples but also captures more short-term and long-term trends, which is particularly crucial for traffic flow prediction. Consequently, as the sample size increases, the model's performance stabilizes and continues to improve. These results validate the robustness of our method when handling different sample sizes, demonstrating that in a big data environment, the method can more effectively learn and predict complex traffic flow patterns.

## Ablation experiment

Our model consists of several modules, including random initialization, graph evolution, and Temporal Convolutional Network (TCN). In our experiments, we conducted ablation studies on our proposed model by removing or modifying some of these modules. Specifically, EG-NODE has four variants: (1) No Initial Matrix Adjustment (NOIMA): This variant removes the initial matrix adjustment. (2) No Grape Evolutional (NOGE): This variant eliminates the evolution of the semantic adjacency matrix. (3) No Front Temporal Convolutional Network (NOFTCN): In this variant, the TCN module before the NODE solver is removed. (4) No Back

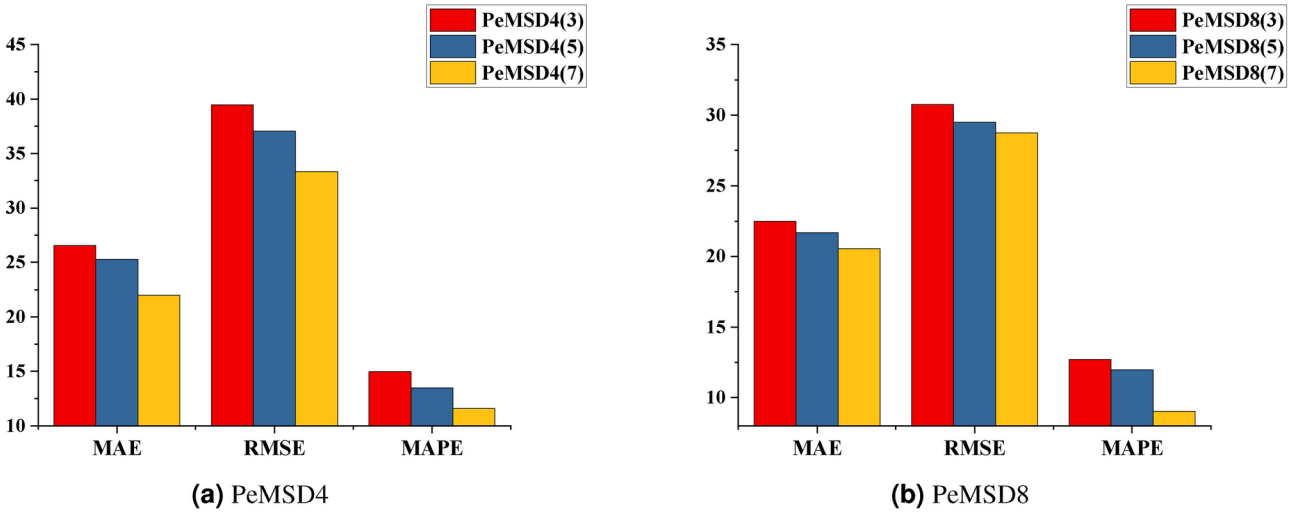



**Fig. 8**. Sample complexity analysis on PeMSD4 and PeMSD8.

Temporal Convolutional Network (NOBTCN): Here, the TCN module after the NODE solver is removed. We evaluated these four model structures, along with our unmodified model, on four datasets. The performance of these models was quantitatively assessed using established metrics: Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and Mean Absolute Percentage Error (MAPE). These metrics provide a comprehensive framework for evaluating and comparing the prediction accuracy of different configurations and datasets.

The experimental results, as depicted in Fig. 9, unequivocally substantiate the superior performance of the EG-NODE model over the NOGE model across four distinct datasets. Specifically, the EG-NODE model achieves a Mean Absolute Error (MAE) that is 1.60 lower than that of NOGE on the PeMSD3 dataset, 2.42 lower on the PeMSD4 dataset, and 2.29 and 2.55 lower on the PeMSD7 and PeMSD8 datasets, respectively. Despite demonstrating superior performance compared to NOGE, the NOIMA model still falls short of the EG-NODE model, thereby highlighting the latter's efficacy.

The comparative analysis of each module further elucidates that NOGE yields the poorest performance, thereby underscoring the significant role that graph evolution operations play in enhancing the model's effectiveness. Furthermore, alterations to the TCN modules in NOFTCN and NOBTCN lead to a detrimental impact on the model's performance, thus validating the indispensable necessity of both TCN modules for optimal performance.

## Visualization

In order to see the role of matrix evolution more intuitively, we conducted visualisation experiments on the PeMSD4 dataset, observing the change of semantic neighbourhood matrices over 25 epochs, and plotting the matrices in a reduced dimensionality using TSNE's method for a more intuitive presentation. It can be clearly seen that the semantic neighbourhood matrix can continuously adjust these semantic relationships as the epoch cycle iterates, making the model more accurate in understanding the associations and connections between the data. The results of the visualization experiments are shown in Fig. 10.

## Speed and efficiency analysis

Inference speed and resource consumption are two critical metrics for machine learning models in real-world applications. Inference speed directly affects a model's real-time responsiveness in practical environments. For large-scale systems that run over extended periods, reducing resource consumption can significantly lower energy usage, thus enhancing the system's sustainability and economic efficiency. To validate our model's performance in these two aspects, we conducted a comprehensive comparison of the inference speed and resource consumption of the EG-NODE model with STGCN, DCRNN, and STGODE models. This analysis helps readers better understand the feasibility of deploying the model in real-world scenarios. As shown in Fig .11, our model's inference speed is significantly faster than STGCN and DCRNN, while it is comparable to STGODE. In terms of resource consumption, specifically memory usage, our model uses significantly less memory than STGCN and DCRNN, while remaining on par with STGODE. Experimental results show that our model occupies less resource consumption and has a faster inference speed.

## Conclusions

In this study, we introduce EG-NODE, a meticulously designed model adept at capturing the complex temporal fluctuations in vehicular traffic. Our methodology has been rigorously validated across five diverse real-world traffic datasets. This comprehensive series of experiments involved benchmarking our model against 16 carefully chosen baseline models. The empirical evidence from this rigorous analysis consistently confirms the superior predictive performance of EG-NODE, substantiating its reliability across various traffic conditions and scenarios. Certainly, our model has room for improvement, especially in the construction of the spatial adjacency matrix.

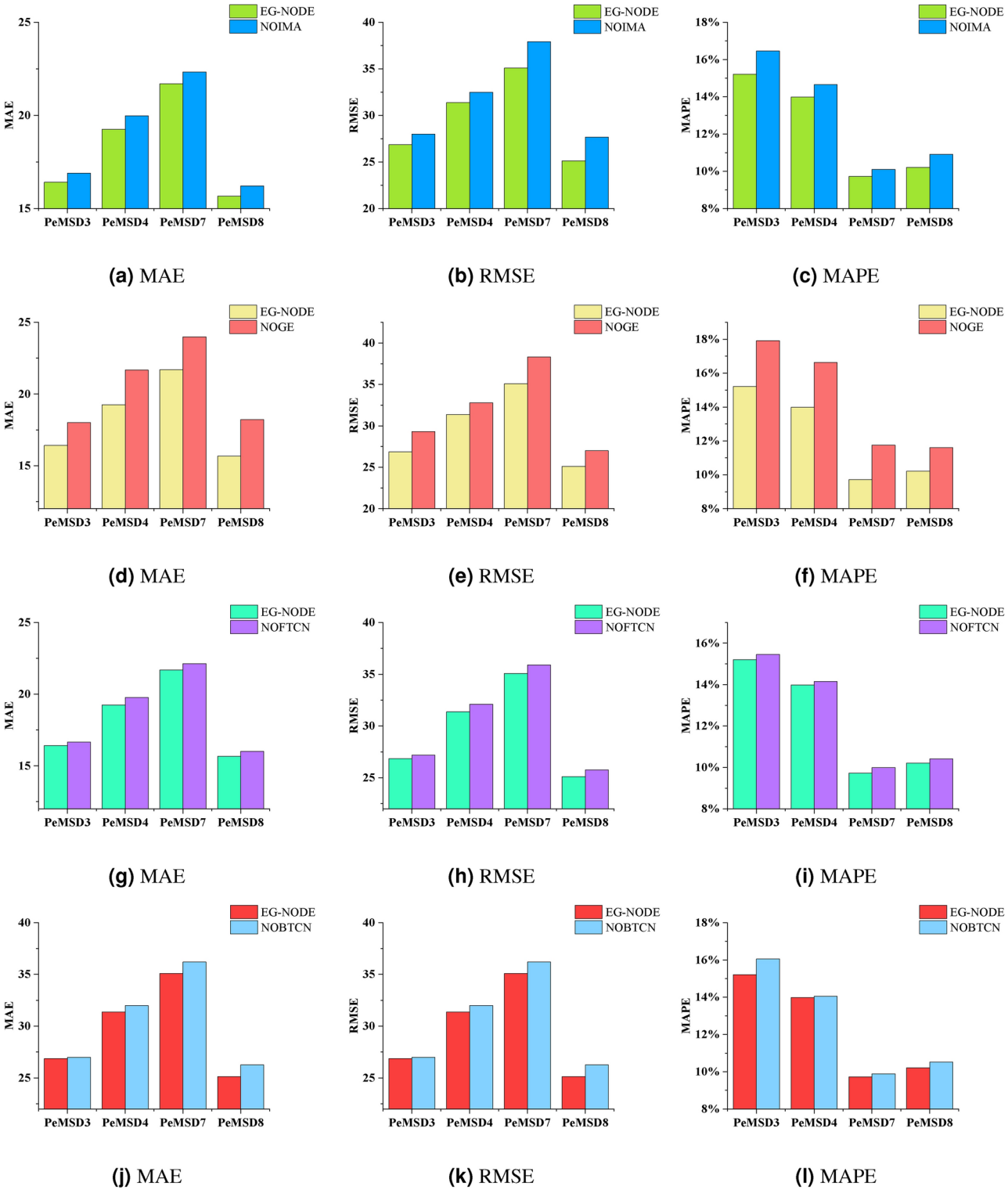



**Fig. 9**. Ablation experimental results.

In our method, the semantic adjacency matrix evolves iteratively during training, allowing it to adapt well to the complex and dynamic nature of traffic flow information. However, our spatial adjacency matrix is predefined and remains unchanged throughout the training process, which is a limitation. In future work, we believe that transforming the construction of the spatial adjacency matrix into a dynamic process, in tandem with the evolving semantic adjacency matrix, could better handle the complex and variable spatiotemporal traffic information.

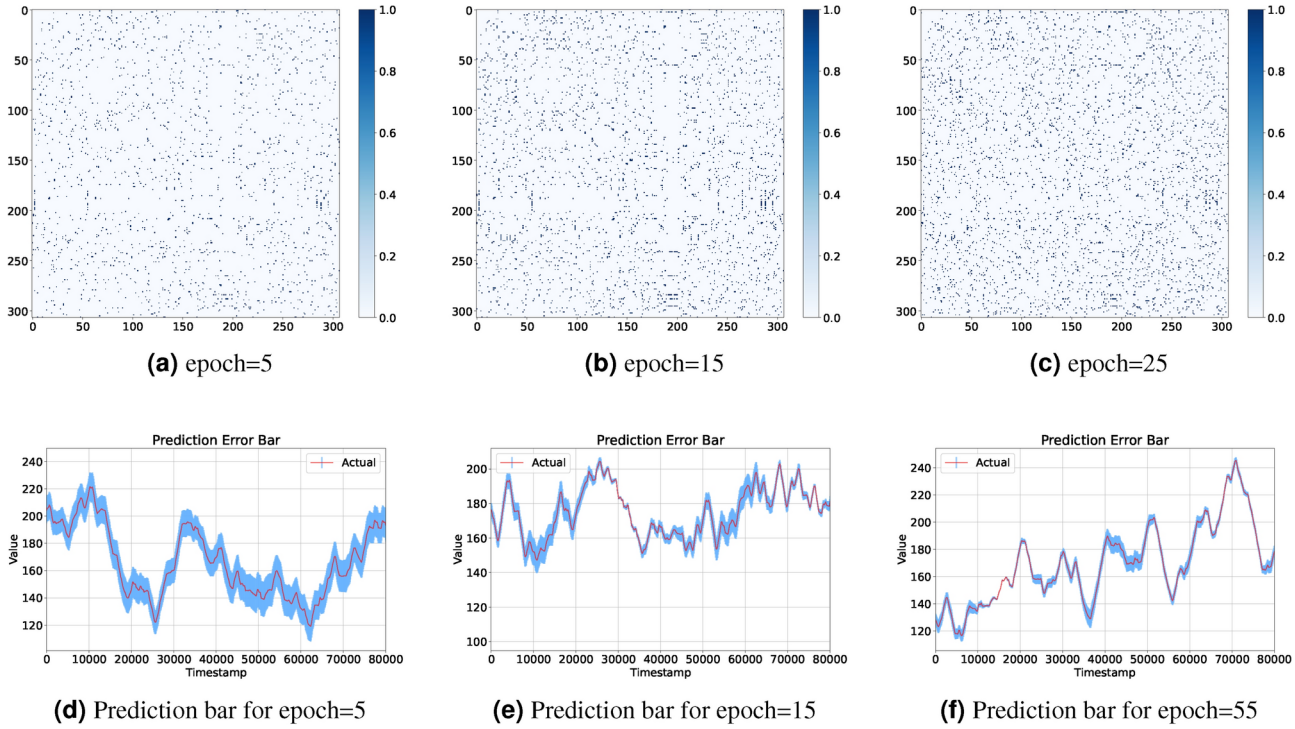



**Fig. 10**. Visualization.

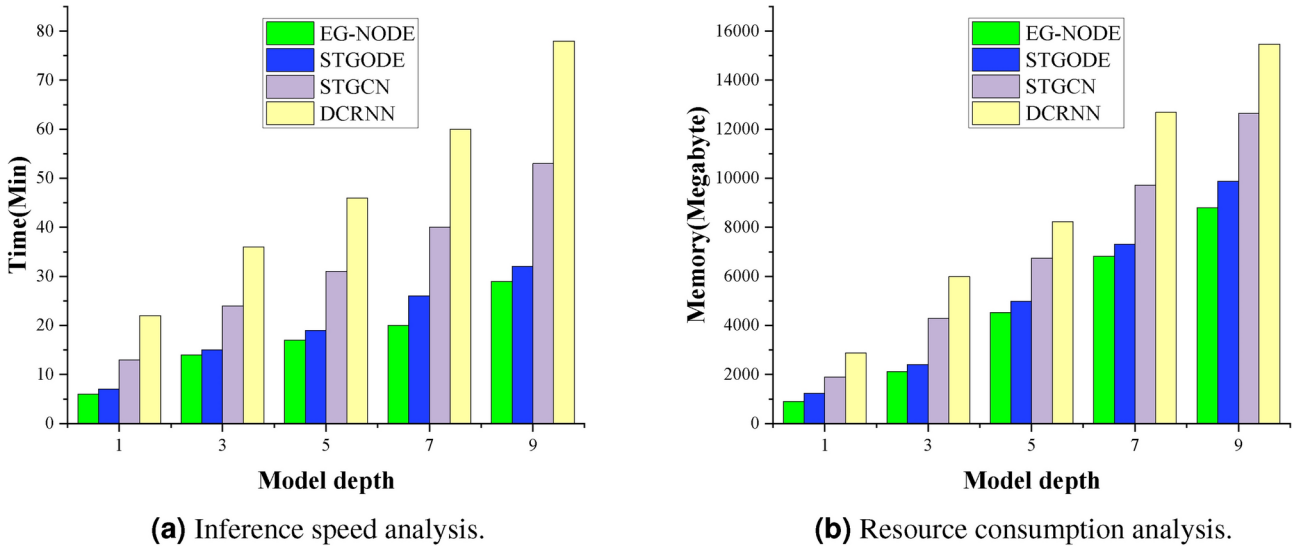



**Fig. 11**. Speed and efficiency analysis on METR-LA.

## Data availability

The datasets used in this study are available on Google Drive. You can access these datasets by following the links below: https://drive.google.com/drive/folders/1g5v2Gq1tkOq8XO0HDCZ9nOTtRpB6-gPe. https://drive.google.com/drive/folders/1g5v2Gq1tkOq8XO0HDCZ9nOTtRpB6-gPe.

Received: 6 June 2024; Accepted: 30 October 2024
Published online: 05 November 2024

## References

1. Butilă, E. V. & Boboc, R. G. Urban traffic monitoring and analysis using unmanned aerial vehicles (UAVS): A systematic literature review. *Remote Sens.* **14**, 620 (2022).

2. Bekkouche, O., Taleb, T. & Bagaa, M. Uavs traffic control based on multi-access edge computing. In *2018 IEEE Global Communications Conference (GLOBECOM)*, 1–6 (IEEE, 2018).
3. Alioua, A., Djeghri, H.-E., Cherif, M. E. T., Senouci, S.-M. & Sedjelmaci, H. Uavs for traffic monitoring: A sequential game-based computation offloading/sharing approach. *Comput. Netw.* **177**, 107273 (2020).
4. Chen, C., Petty, K., Skabardonis, A., Varaiya, P. & Jia, Z. Freeway performance measurement system: mining loop detector data. *Transp. Res. Rec.* **1748**, 96–102 (2001).
5. Box, G. E. P. & Pierce, D. A. Distribution of residual autocorrelations in autoregressive-integrated moving average time series models. *J. Am. Stat. Assoc.* **65**, 1509–1526. https://doi.org/10.1080/01621459.1970.10481180 (1970).
6. Stock, J. H. & Watson, M. W. Vector autoregressions. *J. Econ. Perspect.* **15**, 101–115 (2001).
7. Awad, M., Khanna, R., Awad, M. & Khanna, R. Support vector regression. *Efficient learning machines: Theories, concepts, and applications for engineers and system designers* 67–80 (2015).
8. Peterson, L. E. K-nearest neighbor. *Scholarpedia* **4**, 1883 (2009).
9. Yu, B., Yin, H. & Zhu, Z. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. arXiv preprint arXiv:1709.04875 (2017).
10. Chen, R. T. Q., Rubanova, Y., Bettencourt, J. & Duvenaud, D. K. Neural ordinary differential equations. In Bengio, S. *et al.* (eds.) *Advances in Neural Information Processing Systems*, vol. 31 (Curran Associates, Inc., 2018).
11. Fang, Z., Long, Q., Song, G. & Xie, K. Spatial-temporal graph ode networks for traffic flow forecasting. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 364–373 (2021).
12. Guo, S., Lin, Y., Feng, N., Song, C. & Wan, H. Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. *Proc. AAAI Conf. Artif. Intell.* **33**, 922–929. https://doi.org/10.1609/aaai.v33i01.3301922 (2019).
13. Hildmann, H. & Kovacs, E. Using unmanned aerial vehicles (UAVS) as mobile sensing platforms (MSPS) for disaster response, civil security and public safety. *Drones* **3**, 59 (2019).
14. Khan, M. A., Ectors, W., Bellemans, T., Janssens, D. & Wets, G. Unmanned aerial vehicle-based traffic analysis: A case study for shockwave identification and flow parameters estimation at signalized intersections. *Remote Sens.* **10**, 458 (2018).
15. Iftikhar, S. et al. Target detection and recognition for traffic congestion in smart cities using deep learning-enabled UAVS: A review and analysis. *Appl. Sci.* **13**, 3995 (2023).
16. Aasen, H., Honkavaara, E., Lucieer, A. & Zarco-Tejada, P. J. Quantitative remote sensing at ultra-high resolution with UAV spectroscopy: a review of sensor technology, measurement procedures, and data correction workflows. *Remote Sens.* **10**, 1091 (2018).
17. Box, G. E. & Pierce, D. A. Distribution of residual autocorrelations in autoregressive-integrated moving average time series models. *J. Am. Stat. Assoc.* **65**, 1509–1526 (1970).
18. Williams, B. M. & Hoel, L. A. Modeling and forecasting vehicular traffic flow as a seasonal arima process: Theoretical basis and empirical results. *J. Transp. Eng.* **129**, 664–672. https://doi.org/10.1061/(ASCE)0733-947X(2003)129:6(664) (2003).
19. Sutskever, I., Vinyals, O. & Le, Q. V. Sequence to sequence learning with neural networks. In *Proceedings of the 27th International Conference on Neural Information Processing Systems—Volume 2*, NIPS'14, 3104-3112 (MIT Press, 2014).
20. Shi, X. *et al. Convolutional lstm Network: A Machine Learning Approach for Precipitation Nowcasting*. NIPS'15, 802–810 (MIT Press, 2015).
21. Zhang, J., Zheng, Y. & Qi, D. Deep spatio-temporal residual networks for citywide crowd flows prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31 (2017).
22. Liu, J., Shahroudy, A., Xu, D. & Wang, G. Spatio-temporal lstm with trust gates for 3d human action recognition. In *Computer Vision—ECCV 2016* (eds. Leibe, B., Matas, J., Sebe, N. & Welling, M.) 816–833 (Springer International Publishing, 2016).
23. Nofal, S. Predicting vehicle travel time on city streets for trip preplanning and predicting heavy traffic for proactive control of street congestion. *Sci. Rep.* **14**, 10497. https://doi.org/10.1038/s41598-024-61379-7 (2024).
24. Jiang, W. & Luo, J. Graph neural network for traffic forecasting: A survey. *Expert Syst. Appl.* **207**, 117921. https://doi.org/10.1016/j.eswa.2022.117921 (2022).
25. Li, Y., Yu, R., Shahabi, C. & Liu, Y. Graph convolutional recurrent neural network: Data-driven traffic forecasting. *CoRR*[SPACE]arXiv:1707.01926 (2017).
26. Bai, L., Yao, L., Kanhere, S. S., Wang, X. & Sheng, Q. Z. Stg2seq: Spatial-temporal graph to sequence model for multi-step passenger demand forecasting, vol. 2019-August, 1981–1987 (Macao, China, 2019).
27. Huang, S., Wang, D., Wu, X. & Tang, A. Dsanet: Dual self-attention network for multivariate time series forecasting. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, CIKM '19, 2129–2132, https://doi.org/10.1145/3357384.3358132 (Association for Computing Machinery, 2019).
28. Tran, T., He, D., Kim, J. & Hickman, M. Data-driven traffic incident prediction with hybrid graph-based neural network. In *Australasian Transport Research Forum (ATRF), 42nd, 2021, Brisbane, Queensland, Australia* (2021).
29. Li, J. *et al. Predicting Path Failure in Time-Evolving Graphs* 1279–1289 (Anchorage, 2019).
30. Xing, Z., Huang, M., Li, W. & Peng, D. Spatial linear transformer and temporal convolution network for traffic flow prediction. *Sci. Rep.* **14**, 4040. https://doi.org/10.1038/s41598-024-54114-9 (2024).
31. Huang, X., Jiang, Y., Wang, J., Lan, Y. & Chen, H. A multi-modal attention neural network for traffic flow prediction by capturing long-short term sequence correlation. *Sci. Rep.* **13**, 21859. https://doi.org/10.1038/s41598-023-48579-3 (2023).
32. Guo, S., Lin, Y., Wan, H., Li, X. & Cong, G. Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting. *IEEE Trans. Knowl. Data Eng.* **34**, 5415–5428. https://doi.org/10.1109/TKDE.2021.3056502 (2022).
33. Liu, Z., Lu, X., Liu, W., Qi, W. & Su, H. Human–robot collaboration through a multi-scale graph convolution neural network with temporal attention. *IEEE Robot. Autom. Lett.* **9**, 2248–2255. https://doi.org/10.1109/LRA.2024.3355752 (2024).
34. Huang, X., Zhang, B., Feng, S., Ye, Y. & Li, X. Interpretable local flow attention for multi-step traffic flow prediction. *Neural Netw.* **161**, 25–38. https://doi.org/10.1016/j.neunet.2023.01.023 (2023).
35. Chen, Y., Segovia, I. & Gel, Y. R. Z-gcnets: Time zigzags at graph convolutional networks for time series forecasting. In *Proceedings of the 38th International Conference on Machine Learning, vol. 139 of Proceedings of Machine Learning Research* (eds. Meila, M. & Zhang, T.) 1684–1694 (PMLR, 2021).
36. Song, C., Lin, Y., Guo, S. & Wan, H. Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. *Proc. AAAI Conf. Artif. Intell.* **34**, 914–921 (2020).
37. Wang, X. et al. Traffic flow prediction via spatial temporal graph neural network. *Proc. Web Conf.* **2020**, 1082–1092 (2020).
38. Xu, Z., Lv, Z., Chu, B. & Li, J. Fast autoregressive tensor decomposition for online real-time traffic flow prediction. *Knowl.-Based Syst.* **282**, 111125. https://doi.org/10.1016/j.knosys.2023.111125 (2023).
39. Li, Q., Han, Z. & Wu, X.-M. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32 (2018).
40. Rackauckas, C. & Nie, Q. Differentialequations.jl—a performant and feature-rich ecosystem for solving differential equations in julia. *J. Open Res. Softw.* **5**. https://doi.org/10.5334/jors.151 (2017).
41. Xu, X. et al. Dynamic transformer odes for large-scale reservoir inflow forecasting. *Knowl.-Based Syst.* **276**, 110737. https://doi.org/10.1016/j.knosys.2023.110737 (2023).
42. Choi, J., Choi, H., Hwang, J. & Park, N. Graph neural controlled differential equations for traffic forecasting. *Proc. AAAI Conf. Artif. Intell.* **36**, 6367–6374. https://doi.org/10.1609/aaai.v36i6.20587 (2022).
43. Jin, M. *et al.* Multivariate time series forecasting with dynamic graph neural odes. *IEEE Trans. Knowl. Data Eng.* (2022).

44. Liu, Z., Shojaee, P. & Reddy, C. K. Graph-based multi-ODE neural networks for spatio-temporal traffic forecasting. *Trans. Mach. Learn. Res.* (2023).
45. Saffari, M., Khodayar, M., Khodayar, M. E. & Shahidehpour, M. Behind-the-meter load and pv disaggregation via deep spatiotemporal graph generative sparse coding with capsule network. In *IEEE Transactions on Neural Networks and Learning Systems* 1–15. https://doi.org/10.1109/TNNLS.2023.3280078 (2023).
46. Lei, X., Mei, H., Shi, B. & Wei, H. Modeling network-level traffic flow transitions on sparse data. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 835–845 (2022).
47. Berndt, D. J. & Clifford, J. Using dynamic time warping to find patterns in time series. In *Proceedings of the 3rd International Conference on Knowledge Discovery and Data Mining*, 359–370 (1994).
48. Hewage, P. et al. Temporal convolutional neural (TCN) network for an effective weather forecasting using time-series data from the local weather station. *Soft. Comput.* **24**, 16453–16482 (2020).
49. Li, Y., Yu, R., Shahabi, C. & Liu, Y. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. arXiv preprint arXiv:1707.01926 (2017).
50. Choe, T., Skabardonis, A. & Varaiya, P. Freeway performance measurement system: Operational analysis tool. *Transp. Res. Rec.* **67–75**, 2002. https://doi.org/10.3141/1811-08 (1811).
51. Loudon, N. A review of the experience of thaumasite sulfate attack by the UK highways agency. *Cement Concr. Compos.* **25**, 1051–1058 (2003).
52. Sutskever, I., Vinyals, O. & Le, Q. V. Sequence to sequence learning with neural networks. *Adv. Neural Inf. Process. Syst.* **27** (2014).
53. Wu, Z., Pan, S., Long, G., Jiang, J. & Zhang, C. Graph wavenet for deep spatial-temporal graph modeling. arXiv preprint arXiv:1906.00121 (2019).
54. Guo, S., Lin, Y., Feng, N., Song, C. & Wan, H. Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. *Proc. AAAI Conf. Artif. Intell.* **33**, 922–929 (2019).
55. Huang, S., Wang, D., Wu, X. & Tang, A. Dsanet: Dual self-attention network for multivariate time series forecasting. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, 2129–2132 (2019).


## Acknowledgements

This study is supported by the National Nature Science Foundation of China (61602399, 32302138), and Shandong Provincial Nature Science Foundation, China (ZR2020MF100).

## Author contributions

W.M. designed the research study. Z.C. developed the method and wrote the code. W.M., Z.C., H.C., and M.L. analyzed the results. W.M. and Z.C. wrote the paper. All authors reviewed the manuscript.

## Declarations

## Competing interests

The authors declare no competing interests.

## Additional information

**Correspondence** and requests for materials should be addressed to W.M.

**Reprints and permissions information** is available at www.nature.com/reprints.

**Publisher's note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

**Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2024